Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Воронежский государственный технический университет»
(ФГБОУ ВО «ВГТУ», ВГТУ)

Факультет информационных технологий и компьютерной безопасности
Кафедра систем информационной безопасности
Направление подготовки (специальность) 10.05.03 Информационная безопасность автоматизированных систем
(код и наименование направления подготовки)
Профиль/программа/направленность Обеспечение информационной безопасности распределённых информационных систем

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Дипломная работа
(вид выпускной квалификационной работы – бакалаврская работа, дипломная работа (проект), магистерская диссертация)

Кудинов Кирилл Сергеевич
(фамилия, имя, отчество обучающегося)

Тема: Программно-технический комплекс риск-моделирования кибератак: алгоритмическое и программное обеспечение

Состав выпускной квалификационной работы:

Расчетно-пояснительная записка на 114 страницах
Графическая часть на _____ листах

Расчетно-пояснительная записка к выпускной квалификационной работе:

Заведующий кафедрой _____ (подпись) (А.Г. Остапенко) (инициалы, фамилия)

Руководитель _____ (подпись) (Л.В. Паринова) (инициалы, фамилия)

Консультанты:
по _____ (наименование раздела, подпись) ( ) (инициалы, фамилия)
по _____ (наименование раздела, подпись) ( ) (инициалы, фамилия)
по _____ (наименование раздела, подпись) ( ) (инициалы, фамилия)
по _____ (наименование раздела, подпись) ( ) (инициалы, фамилия)
по _____ (наименование раздела, подпись) ( ) (инициалы, фамилия)

Обучающийся _____ (подпись) К.С. Кудинов (инициалы, фамилия)

«02» 07 2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГТУ», ВГТУ)

Факультет информационных технологий и компьютерной безопасности
Кафедра систем информационной безопасности
Направление подготовки/специальность 10.05.03 Информационная безопасность автоматизированных систем
(код, наименование)
Профиль/программа/направленность Обеспечение информационной безопасности распределённых информационных систем

Утверждаю _____
(дата)
Зав.кафедрой _____
(подпись)

ЗАДАНИЕ
ПО ПОДГОТОВКЕ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

Дипломная работа
дипломная работа (проект), бакалаврская работа, магистерская диссертация

Обучающемуся Кудинову Кириллу Сергеевичу
фамилия, имя, отчество

1. Тема: Программно-технический комплекс риск-моделирования кибератак: алгоритмическое и программное обеспечение

ФГБОУ ВО «ВГТУ», ВГТУ
по заданию какой организации выполняется работа

приказ №01-1-10/3-2979 от 25.06.2025
утверждена приказом по университету

2. Срок сдачи обучающимся выпускной квалификационной работы 02.07.25

3. Исходные данные

4. Краткое содержание выпускной квалификационной работы
Раздел 1 Алгоритмические основы риск-моделирования кибератак
перечень подлежащих разработке в выпускной квалификационной работе вопросов
Раздел 2 Алгоритмическая часть комплекса риск-моделирования кибератак
Раздел 3 Проектирование программного обеспечения риск-моделирования кибератак

5. Перечень графического материала (с точным указанием обязательных чертежей по разделам

6. Консультанты (с указанием относящихся к ним разделов)

График выполнения выпускной квалификационной работы

№ п/п	Наименование разделов выпускной квалификационной работы	Срок выполнения этапов работы
1	Введение	22.03 – 22.05
2	Алгоритмические основы риск-моделирования кибератак	22.03 – 22.05
3	Алгоритмическая часть комплекса риск-моделирования кибератак	23.05 – 31.05
4	Проектирование программного обеспечения риск-моделирования кибератак	01.06 – 24.06
5	Заключение	25.06 – 02.07

Руководитель _____
(подпись)

Дата выдачи задания 22.03.25

Задание принял к исполнению 22.03.25
(дата)

Подпись обучающегося _____

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Воронежский государственный технический университет»

АННОТАЦИЯ
выпускной квалификационной работы

Обучающегося Кудинова Кирилла Сергеевича

Тема ВКР: Программно-технический комплекс риск-моделирования кибератак: алгоритмическое и программное обеспечение

Характеристика выпускной квалификационной работы

Актуальность исследования: современный инструментарий по расчету рисков кибератак в информационной безопасности требует автоматизации анализа уязвимостей и сценариев кибератак в условиях роста числа CVE и сложности атакующих техник. Ручной расчёт рисков становится трудоёмким и неэффективным, что требует разработки новых алгоритмов и программных решений для ускорения и точности оценки.

Задачи, решаемые в ВКР: систематизация подходов к моделированию кибератак; разработка алгоритмов агрегации данных из баз CAPEC, ATT&CK, CWE и CVE; реализация алгоритмов расчёта риска единичных, множественных и многовекторных атак; реализация алгоритмов по управлению показателями риска для кибератак; создание программного комплекса для оценки риска сценариев атак с их визуализацией.

Структура ВКР: исследование алгоритмических возможностей риск-моделирования кибератак; разработка алгоритмов автоматизированного обновления связанной базы данных, расчета риска для единичных, множественных и многовекторных атак, управления риском; разработка программного обеспечения с учетом разработанных алгоритмов.

Краткая характеристика полученных результатов: Разработан программно-технический комплекс риск-моделирования кибератак, реализующий автоматический сбор данных, построение сценариев атак и вычисление показателей риска. Предложены и протестированы алгоритмы, подтверждена корректность расчётов на основе реальных CVE.

Автор ВКР _____ Кудинов Кирилл Сергеевич

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 7
1 Алгоритмические основы риск-моделирования кибератак 13
1.1 Модели представления кибератак и уязвимостей 13
1.2 Алгоритмы агрегации данных об уязвимостях и атаках для построения моделей 15
1.3 Алгоритмические подходы к оценке рисков кибератак 17
2 Алгоритмическая часть комплекса 19
2.1 Построение связанной модели 19
2.2 Алгоритмы расчёта риска 21
2.2.1 Алгоритм расчета риска для единичных атак 21
2.2.2 Алгоритм расчета риска для множественных атак 40
2.2.3 Алгоритм расчета риска для многовекторных атак 47
3 Проектирование программно-технического комплекса 52
3.1 Общее описание архитектуры комплекса 52
3.2 Выбор технологий и инструментов 55
3.3 Компоненты пользовательского интерфейса 68
3.3.1 Ввод и обработка идентификаторов уязвимостей 71
3.3.2 Построение сценариев атак 73
3.4 Модуль визуализации 75
3.4.1 Вывод графов 75
3.4.2 Вывод графиков и взаимодействие с ними 77
3.4.3 Вывод списков информации 81
3.6 Ключевые направления для будущей доработки 81
ЗАКЛЮЧЕНИЕ 85

СПИСОК ЛИТЕРАТУРЫ 87
ПРИЛОЖЕНИЕ 95
1. Структура данных 95
2. Лингвистическое обеспечение 96
3. Пример применения программного комплекса риск-моделирования кибератак 96
3.1 Работа функционала по обновлению и вспомогательных инструментов 96
3.2. Пример работы по расчёту риска для кибератак 99
4. Программный код базовых модулей функции 105

ВВЕДЕНИЕ

Актуальность темы исследования. Рост киберпреступности в современном информационном пространстве непрерывно растет [1-9], причем растет не только само их число, но и процентная доля относительно общего числа преступлений. Согласно последним сводкам в Российской Федерации за 2024 год, общая доля киберпреступлений от общей массы составляет уже около 40%, а это порядка 765,4 тысяч инцидентов, а ущерб составил порядка 116 миллиардов рублей [31, 37-40].

Ключевые факторы, подтверждающие актуальность темы:

1) Рост доли и числа киберпреступлений.

Сравнив эту долю со статистикой за прошлые годы [37-40]:

- в 2020 это 25% от общего числа преступлений, что составило около 363 тысяч преступлений;
- в 2021 это 25,8% от общего числа преступлений, что составило около 271,1 тысячу преступлений;
- в 2022 это 26, 5% от общего числа преступлений, что составило около 334,1 тысячи преступлений;
- в 2023 это 35% от общего числа преступлений, что составило около 677 тысяч преступлений.

2) **Рост числа уязвимостей.**

Статистика прямо указывает на то, что рост преступлений имеет направленный вверх вектор, причем его множитель не понятен, он имеет хаотичный характер. Статистические данные с порталов уязвимостей так же информируют нас о стремительных показателях роста, так согласно базе данных NVD количество зарегистрированных уязвимостей составило [18]:

- 17492 за 2018 год, однако еще в 2017 году их было всего 6201, рост почти в три раза.
- 17030 за 2019 год, уровень остался приблизительно на том же уровне;
- 20601 за 2020 год, более чем на 3 тысячи больше чем в прошлом;
- 23073 за 2021 год, стабильный рост за последние два года;

- 26708 за 2022 год, все такой же стабильный рост за последние три года;
- 29763 за 2023 год, тенденция набирает приблизительно понятный характер;
- 38365 за 2024 год, что олицетворяет стремительный рост, не такой как в переходе с 2017 на 2018 год, но если рассмотреть количественную составляющую, то становится видно, что пик сравним с ранее описанным периодом;
- 17193 на момент начала июня 2025 года, данное число не может пока что освидетельствовать о чем либо, однако назвать такое число незначительным так же нельзя.

Данный рост открывает все новые возможности для реализации различных сценариев атак, которые приводят различным рискам для информационных структур компаний, что отражает статистика третьего фактора актуальности темы исследования.

3) Статистические показатели видов атак на компании.

Согласно отчету компании Solar JSOC за 2024 год процентные части на различные виды киберпреступлений были следующими:

- эксплуатация уязвимостей — 12%;
- срабатывание сигнатуры сенсоров SOC — 10%;
- НСД к ИС и сервисам — 8%;
- сетевые атаки — 7%;
- использование нелегитимного ПО — 5%;
- компрометация УЗ — 4%;
- компрометация ИС по информации TI — 3%;
- веб-атаки — 2%;
- остальное — 9%.

В совокупности это все подчеркивает то, что эксплуатация уязвимостей становится все более популярным направлением для атак, которое для специалиста информационной безопасности убудут являться рутинным делом, ведь ручной подсчет рисков для векторов уязвимостей, состоящих из нескольких уязвимостей будет занимать достаточно долгое время.

Работа посвящена разработке алгоритмов и реализации на их основе программно-технического комплекса риск-моделирования кибератак. Основной упор делается на алгоритмы сбора и агрегации данных, для их подготовки под работу с алгоритмами расчета единичных, множественных и многовекторных атак. Комплекс будет включать в себя блоки автоматизированного обновления баз данных, информирования об уязвимостях, построения иерархии CAPEC→MITRE ATT&CK→CWE→CVE и расчёта сценариев атак.

Степень разработанности темы исследования.

Риск моделирование активно изучается и описывается как в литературе, так и в статьях с отчетами ведущих компаний в области информационной безопасности, ведущими среди них являются [42]:

- CAPEC, они описывают общие шаблоны атак, например SQL-инъекции;
- MITRE ATT&CK, в их базах содержится продолжение идеи CAPEC, а именно тактики реализации шаблонов, которые включают в себя различные техники;
- CWE, содержит общее описание слабости программного обеспечения;
- CVE в свою очередь является точечной базой, которая указывает на конкретную уязвимость в программном обеспечении.

Несмотря на тесное взаимодействие между собой этих мастодонтов в области информационной безопасности, современные решения используют их данные либо узконаправлено, либо не в полном объеме.

Недостатки текущих решений:

- ограниченность, UML-модели и деревья угроз, которые реализуются в риск моделировании имеют ограниченную масштабируемость, так как при большом числе элементов они становятся нечитаемыми;
- статические метрики, такие как у CVE могут лишь поверхностно описать риски, связанные с ней, они не учитывают контекст атаки и ее динамику, что подтверждает статистика о том, что 20% уязвимостей эксплуатируются после публикации, но в расчетах и данных этих метрик указателей на это нет;
- фрагментарность информации, большинство технических решений разрозненно рассматривает сценарии атаки, не учитывая их полною структуру,

исследование чаще всего ведется либо поверхности, либо точечно, хотя полный анализ возможен через использование последовательности этих действий, но в таком случае, он очень трудоемок;

- ручная трудоемкость, как и ранее упоминалось, процесс расчёта риска для различных кибератак с учетом ныне представленных решений является долгим и трудоемким, а в таких условиях невелик шанса человеческого фактора.

Эти недостатки в полной мере подчеркивают потребность нынешней информационной защиты в массивных инструментах по расчету рисков для различных атак.

Объект исследования: сети и системы информационной безопасности, подверженные различным сценариям атак, их шаблоны, тактики и техники, а также связанные с ними классы уязвимостей и наполнители этих классов (сами уязвимости).

Предмет исследования: алгоритмы расчёта риска сценариев атак, их визуализации, построения связей и программный комплекс, основанный на этих алгоритмах.

В ходе анализа между характеристиками объекта и предмета был выявлен следующий **ряд противоречий,** а именно:

- **между динамикой изменения ландшафта угроз и периодичностью обновления моделей**;

- **между ручным подходом и требованиям к максимальной автоматизации, с учетом роста числа уязвимостей и уменьшением времени на принятие решения**;

- **между необходимостью анализа многовекторного анализа и ограниченностью существующих моделей** в этом, которые в большей степени рассматривают разовые или статические сценарии.

Основной **целью** данной работы является разработка эффективных алгоритмов по расчету рисков для различных типов атак и реализации возможностью их управления, а также реализации программного комплекса на основе них с модулями визуализации.

Для реализации поставленной цели в работе решаются следующие ключевые **задачи**:

1) обзор представленных решений для выбора направления реализации цели работы;

2) разработка алгоритмов для:

- нахождения соответствий в базах данных для построения связей;
- автоматизированного обновления числовых метрик необходимых для расчетов рисков кибератак;
- разработка алгоритмов для управления показателями рисков кибератак.

3) программная реализация поставленных в задачи алгоритмов с возможностью их визуализации.

Выносимые на защиту результаты. В соответствии с вышеупомянутыми задачами работы на защиту выставлены следующие результаты:

1) алгоритмы нахождения соответствий в базах данных для построения связей, алгоритмы автоматизированного обновления числовых метрик необходимых для расчетов рисков кибератак, алгоритмы для управления показателями рисков кибератак;

2) программно-технический комплекс, реализованный на основе разработанных алгоритмов с модулями визуализации;

3) экспериментальные данные, полученные при тестировании данного программно-технического комплекса.

Новизна исследования:

1) впервые сделана комплексная иерархия, глубоко описывающая шаблоны атак CAPEC для дальнейшего анализа сценариев атак внутри него;

2) разработаны более совершенные алгоритмы, позволяющие более точно просчитывать показатели рисков для кибератак;

3) предложены новые алгоритмы по управлению показателями риска в рамках множественных и многовекторных атак.

Теоретическая ценность данного исследования заключается в системном подходе к анализу киберугроз и методов оценки их риска. Работа вносит вклад в развитие научных знаний в следующих аспектах:

1) интеграция методов комплексного анализа кибератак, через новые алгоритмы расчета их риска, управления им и получение дополнительной информации через вспомогательные инструменты;

2) уточнение критериев оценки риска кибератак через алгоритмы оценки, учитывающие не только один показатель CVSS;

3) предложена иерархичная модель угроз, основанная на нахождении соответствий в смежных базах данных.

Практическая ценность работы состоит в том, что её результаты могут быть использованы:

1) в научных и учебных целях, для глубокого изучения различных кибератак;

2) в индустрии информационной безопасности, для интеграции с другими средствами защиты, с целью создания комплекса с большим функционалом;

3) в исследовательских задачах, с целью доработки комплекса и достижения его большей автоматизации.

1 Алгоритмические основы риск-моделирования кибератак

1.1 Модели представления кибератак и уязвимостей

В практике информационной безопасности используют различные модели для описания атак на различные ресурсы защищаемых систем. Модели отличаются в основном глубиной исследования и своим назначением [10, 12].

Рассмотрим модели представления кибератак, первыми на очереди будут последовательные модели, примером таковой является Cyber Kill Chain, она описывает атаку, как последовательность из семи фаз: разведки, подготовки, доставки, эксплуатации, установки средства контроля, командования и управления, действия по целям [13, 10]. Данная поэтапная структура облегчает задачу построения защиты для системы, так как инцидент будет поделен на жизненные циклы и контрмеры будут вводиться не на всю атаку целиком, а на поочередно, на каждый ее этап [10].

В эту же копилку попадают матрицы тактик и техник MITRE ATT&CK, они представляют собой тактики (цели злоумышленника, по типу разведки, проникновения и т.д.) и техники (конкретные приёмы, направленные на достижение цели) в табличном формате. Строки таблицы в данной модели задают прием, через который будет реализован данный этап атаки, а столбцы задают этапы (ранее упомянутые цели), такая классификация позволяет оперативно сегментировать вектор атаки, что так же как и у прошлой модели упростит противоборство, а еще и позволит вести классификацию инцидентов [14, 10].

Следующими рассмотрим уже другой класс, а именно процессно-ориентированные руководства, ярким примером таковых выступает NIST SP 800-115 «Technical Guide to Information Security Testing and Assessment». Данное руководство регламентирует этапы тестирования информационной безопасности защищаемой системы. Регламенты содержат в себе следующие этапы: планирование, сбор сведений, выявление уязвимостей, эксплуатацию и отчётность [15]. Обычно при использовании этого руководства специалисты в области информационной безопасности не ограничивают себя его рамками, например компания по

сертификации в области информационной безопасности EC-Council включает дополнительную сертификацию CEH (Certified ethical hacker). Данная сертификация дополняет ранее упомянутое руководство каталогом техник сетевого, прикладного и беспроводного пентестинга, что приближает точность оценок эксперта к действительным и позволяет более грамотно расставлять приоритеты при построении защитных мер [16, 10].

Атаки на информационные структуры тянутся со времен создания электронно-вычислительных машин, за это время у атак образовывались какие-то паттерны, шаблоны, именно такая шаблонная модель CAPEC (Common Attack Pattern Enumeration and Classification) и была создана. В настоящее время она поддерживается ранее упомянутой MITRE и содержит свыше 600 шаблонов атак с описанием предпосылок, процесса эксплуатации и последствий. CAPEC использует системы обеспечения информационной безопасности, такие как SIEM (Security Information and Event Management) и EDR (Endpoint Detection and Response) для автоматического сопоставления обнаруженных активностей с известными сценариями нападения, так как системы выполняют задачи, то итоговый результат будет обладать более высокой точностью [17]. Стоит отметить, что по иерархии шаблоны CAPEC находятся на самой вершине и включают в себя ранее упомянутые матрицы тактик MITRE ATT&CK, так как могут быть описаны через них.

В ходе моделирования комплекса было принято решение опираться на шаблоны CAPEC и тактики с техниками MITRE ATT&CK, но они составляют не полную иерархию атаки, они описывают ее поверхностно, им не хватает детальности, а именно описания уязвимостей через которую реализуется тот или иной этап атаки. [10, 14, 17, 18]. Уязвимости тоже не так просто устроены, и каждая из них можно классифицировать в группы, именно для этого была создана классификация CWE (Common Weakness Enumeration), которая систематизирует корневые ошибки проектирования и реализации ПО [19]. Каждая запись CWE описывает типичный шаблон уязвимости, предпосылки возникновения, эти данные помогают стандартизировать анализ рисков при разработке и аудите, однако рекомендации по устранению будут носить шаблонный характер, возникает нехватка конкретики [9].

Последним уровнем в иерархии должно стать описание чего-то конкретного, той самой точки, устранив которую удастся повысить стойкость системы к шаблону CAPEC. Такие точечные описания об уязвимости находятся в базе CVE (Common Vulnerabilities and Exposures), компании NVD, она задаёт уникальные идентификаторы уязвимостей, одна уязвимость – один идентификатор. Для каждой уязвимости рассчитываются метрики с помощью калькуляторов, в данной базе используется CVSS (Common Vulnerability Scoring System), он отражает числовые метрики воздействия на конфиденциальность, целостность и доступность [17, 18].

Выбор конкретной модели зависит строго от поставленных на анализ задач, доступности данных и целей применения, а также от стратегии планирования мер защиты и автоматизации реагирования на инциденты. Модель рассматриваемая нами в работе будет использовать марковские цепи, то есть мы будем моделировать ситуацию сценария, последовательность которого известна нам [11].

1.2 Алгоритмы агрегации данных об уязвимостях и атаках для построения моделей

Для построения какой-либо модели нужны данные, но когда их источники децентрализованы, то возникает проблема их сбора. Агрегация данных помогает решить эту проблему. Процесс основан на сборе данных в разнородных источниках. Он берет начало с извлечения информации из CAPEC, MITRE ATT&CK, CWE и NVD (CVE). Данные нормализуют в единый формат (например, JSON или CVRF), сопоставляя поля «id», «name», «description» и «related_items» по каждому каталогу [17, 14, 19, 18].

Для удаления дубликатов и связывания записей применяют блочное разбиение по ключевым атрибутам (CAPEC-id, Technique ID, CWE-id, CVE-id) с последующей точной сверкой строковых полей для повышения полноты маппинга [20].

Группировка данных по таксономии моделей (CAPEC→ATT&CK→CWE→CVE) обеспечивает иерархическую агрегацию метрик. На первом уровне собирают шаблон CAPEC, на следующем набор техник ATT&CK,

затем — число CWE-классов на технику, и, наконец, характеристики CVE-уязвимостей (согласно калькулятору CVSS) так же, на каждую CWE [17, 14, 19, 18].

Для оперативного мониторинга потоковых событий об уязвимостях и атаках используют системы на базе Apache Kafka и Flink, которые обрабатывают данные в скользящих окнах длительностью W минут [25]. В каждом таком окне вычисляют ключевые метрики:

- число уникальных CVE-атак [21];
- распределение применённых ATT&CK-техник [20];
- частоту появления различных CWE-классов уязвимостей.

Подобный подход позволяет обновлять данные в базе мгновенно и хранить лишь компактные структуры для быстрых запросов по реальному времени [25].

Глубокий анализ и подготовку обучающих выборок проводят в пакетном режиме на кластерах Spark или Hadoop MapReduce. Картографический этап конвертирует исходные записи CAPEC/ATT&CK/CWE/CVE в пары «ключ → метрика», а этап свертки предварительно обработанных данных суммирует, считает средние и формирует итоговые показатели по таксономии моделей [26]. Результат сохраняют в едином хранилище для последующей генерации моделей.

Контроль качества данных осуществляется двумя метриками:

- покрытием — долей записей с непустыми ключевыми атрибутами;
- полнотой — отсутствием пропусков в основных полях.

Если покрытие или полнота падают ниже 95 %, система сигнализирует об этом и запускает процедуру ручного обогащения источников [23, 24].

Итоговый набор данных включает следующие иерархические признаки:

- число ATT&CK-техник, соотнесённых с конкретным CAPEC;
- количество CWE-классов внутри техники;
- количество уязвимостей (CVE) внутри каждой CWE;
- метрики CVSS калькулятора для каждой CVE.

Все алгоритмические блоки выполнены модульно. Благодаря этому какие-либо изменения в базах данных целевых источников не вызовут потребностей в доработке или перестройке программной части модуля агрегации.

1.3 Алгоритмические подходы к оценке рисков кибератак

В алгоритмических подходах к оценке рисков кибератак выделяют несколько ключевых семейств, различающихся по требованиям к данным, способам обработки неопределённости и вычислительным затратам.

Первым выделим, из-за своей наибольшей распространенности оценку по CVSS калькулятору, так называемые (Scoring-модули). Оценка уязвимости измеряется единой метрикой в интервале от 0 до 10 на основе компонентов конфиденциальности, целостности и доступности [18].

Следующими обозначим двумерные матрицы рисков (Risk matrix). Матрица располагает события в сетке «вероятность × влияние» по экспертным оценкам. Применение такого подхода удобно, когда стоит задача быстрой визуализации приоритетов, но субъективна и плохо масштабируется при большом числе сценариев [12].

Малоизученными и самыми полезными в нынешних реалиях будут являться подходы, взятые из теории вероятностей. В качестве таких подходов можно брать биномиальное и мультиномиальное (так же называемое как полиномиальное или многочленное) распределения.

Первое моделирует число успешных атак k из n попыток с вероятностью успешной реализации уязвимости p. Оно позволяет вычислить вероятность меньшего или равного k числа компрометаций P ($\geq$ k) за заданный период и заложить ресурсы реагирования [27].

Второе распределение обобщает биномиальную модель на k типов атак с вероятностями $p_1 \ldots p_k$, описывая совместное распределение числа срабатываний каждого вектора за n испытаний. Пригодно для прогнозирования нагрузки на разные подсистемы информационной безопасности [28].

Для оценки рисков так же применимы подходы, связанные с квантильными методами (VaR и CVaR), Value-at-Risk (q-квантиль распределения потерь) показывает максимальный ущерб при доверии q, а Conditional VaR оценивает средний ущерб

сверх этой квантили для редких, но критичных инцидентов [29]. Данный подход может быть удобен при реализации управления риском.

Далее можно отметить Monte Carlo-симуляции, подход состоит в том, что генерируют тысячи траекторий по выбранной стохастической модели (биномиальной, марковской или графовой) и на основании эмпирического распределения рассчитывают квантили VaR или CVaR с доверительными интервалами [30].

Следующими могут быть применимы Байесовские сети. Они представляют собой графо-вероятностные модели. Модели кодируют причинно-следственные зависимости между уязвимостями и шагами атаки, затем с помощью алгоритма распространения доверия belief propagation рассчитывают апостериорные шансы компрометации целевых активов при полиномиальной сложности в разреженных сетях [11].

Одним из последних можно выделить подход, основанный на машинном обучении. Супервизированные модели (логистическая регрессия, Random Forest, градиентный бустинг) обучаются на исторических инцидентах с данными, взятыми из CAPEC, MITRE ATT&CK, CWE, CVSS и сетевой телеметрии. Не смотря на потенциальную высокую точность, данный подход требует больших размеченных выборок и активного развития направления исследований в области искусственного интеллекта в этой среде для интерпретации результатов [11].

Заключительными отмечу гибридные архитектуры. Они совмещают простые Scoring-модули и распределённые методы (биномиальные и мультиномиальное распределения, квантильные метрики, Монте Карло) с Байесовскими сетями и ML-моделями для баланса между скоростью, точностью и оправданностью прогнозов.

В сравнительных вариациях наибольшее предпочтение оказывают подходы, которые требуют меньшего объема вычислительной мощности, так как для ресурсами для реализации LM моделей обладают только крупные корпорации. Поэтому приоритетными выступают CVSS (Scoring), простой и гибкий для дальнейших расчетов, а также биномиальное и мультиномиальное распределения с квантилями из-за своей интерпретируемости и прозрачности в применении.

2 Алгоритмическая часть комплекса

2.1 Построение связанной модели

Первым шагом на пути выполнения поставленных в работе задач, является заготовка фундамента для расчетов для кибератак. Данным фундаментом является построение связанной базы данных. В качестве структуры базы данных будет взят невзвешенный граф, он опишет зависимости CAPEC → MITRE техники → CWE → CVE для каждого CAPEC. Для построения связанного невзвешенного графа такой структуры нам понадобится следующие источники: MITRE CWE, MITRE ATT&CK, CAPEC и NVD. MITRE CWE предоставляет актуальные данные по всем существующим CWE, причем база обновляется ежедневно, что позволит в дальнейшем поддерживать актуальность работы программы, в этом источнике мы найдем связи между CWE и CAPEC. MITRE ATT&CK в схеме нахождения связей не участвует, этот источник необходим для дополнительного информирования о технике, так как связи, из базы источника CAPEC описывают связь между CAPEC и техниками MITRE, но предоставляют ничего кроме номерного идентификатора. Оставшийся источник NVD описывает связь между CWE и CVE, а также предоставляет сведения по последнему, которые в дальнейшем пригодятся. Автоматизированный алгоритм построения связей описан на рисунке 1 [14, 17-19].

После загрузки баз данных со всех источников происходит их распаковка и последовательная обработка. Первым из файлов CVE происходит перебор каждой из них по отдельности, где из поля Problem Types извлекается идентификатор CWE, а затем в базу данных записывается CVE и связанные с ней CWE. Как только все файлы с CVE будут обработаны случится переход к файлам CWE, в них так же будет происходить поиск, но поля Attack Patterns, которое укажет на связанную с CWE технику MITRE ATT&CK. В базу данных ко всем CWE будут добавлены записи о связанных с ними техниках после заключения анализа файла. После этого действия начнется предпоследняя последовательность, в ней будет проанализирован файл с базой CAPEC. Последовательность действий аналогична прошлым, так же для каждого экземпляра CAPEC будет выбрано поле Taxonomy Name, которое будет

характеризовать связанную с текущим CAPEC одну или несколько тактик, после чего на основе соответствия в базу данных для каждой техники будет добавлен связанный с ней CAPEC. Таким образом мы получаем связанную базу данных, но для соблюдения поставленной иерархии мы отзеркалим ее, тем самым садаптируем базу под вид зависимости CAPEC → MITRE техники → CWE → CVE [14, 17-19].

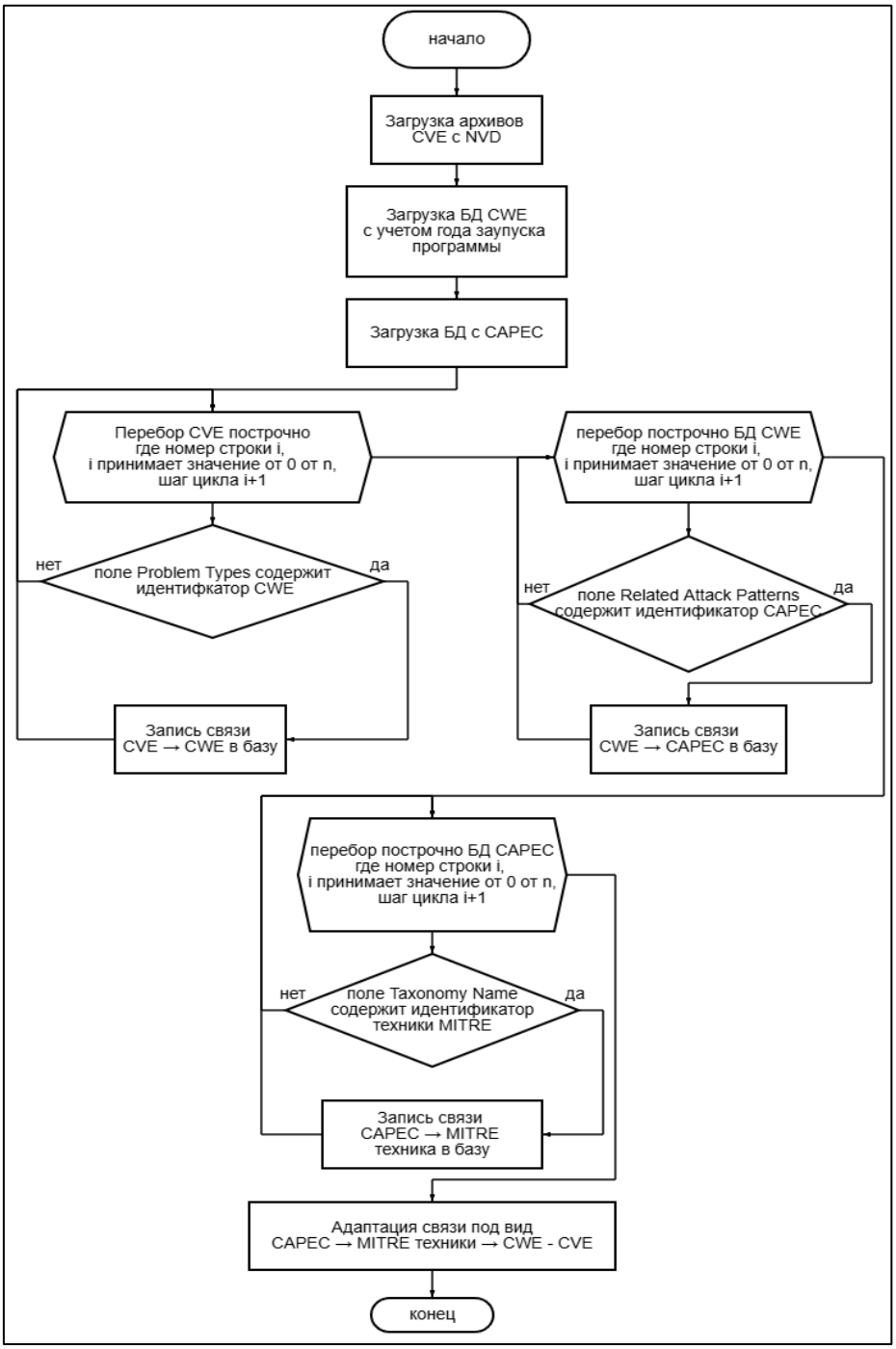


Рисунок 1 – Алгоритм автоматизированной сборки связей CAPEC → MITRE техники → CWE → CVE

Структура БД источников устроена таким образом, что одна строка хранит информацию об одном объекте (CVE, CWE, CAPEC или технике MITRE). Алгоритм позволяет позволять составить уникальную базу данных, не имеющую аналогов. Дальнейшее пояснение структуры базы данных будет затронуто в приложении выпускной квалификационной работы.

2.2 Алгоритмы расчёта риска

2.2.1 Алгоритм расчета риска для единичных атак

Перед тем как перейти к каким-либо расчетам показателей риска, необходимо получить все данные, которые нужны для этого. Для наполнения такими данными решено было использовать набор калькуляторов, а именно CVSS 2, 3.1 и 4 версии, а также калькулятор OWASP и калькулятор EPSS, схему, а наполнения этими данными представлена на рисунке 2.

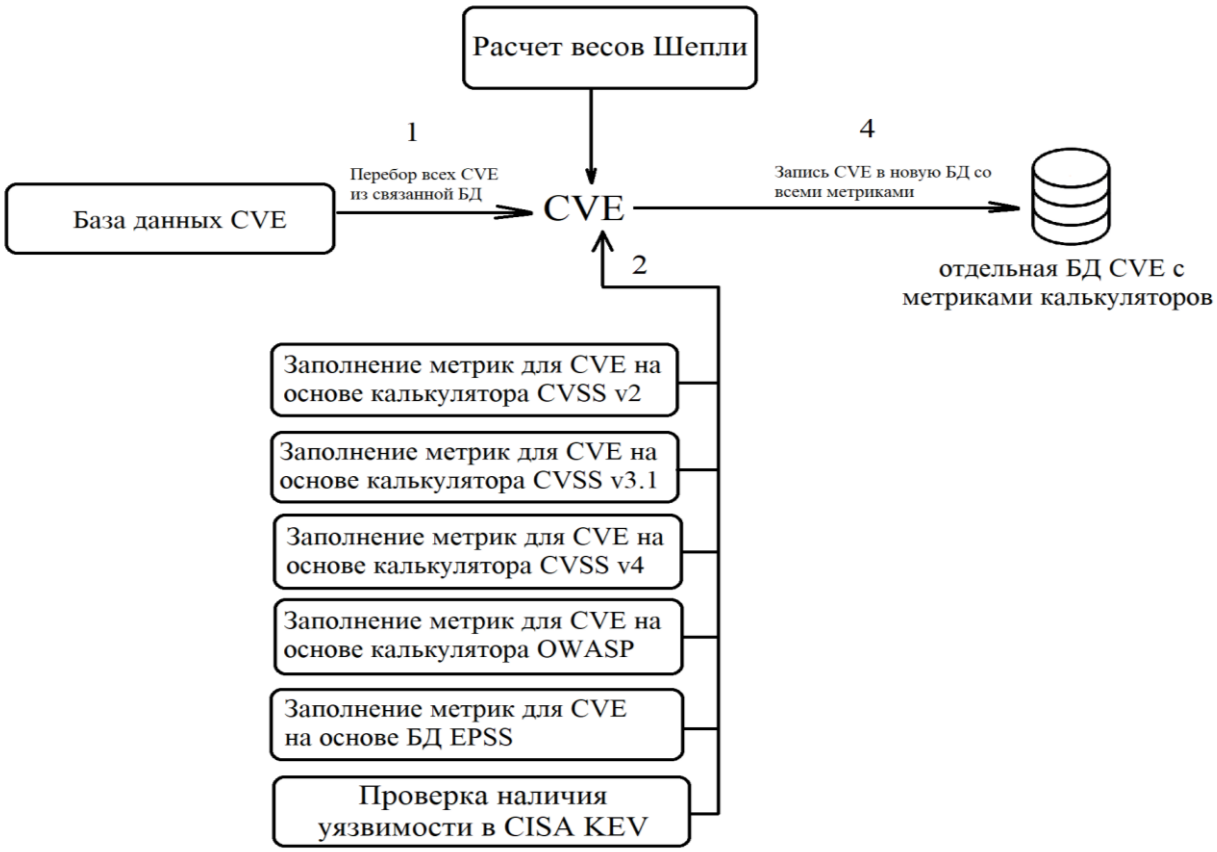


Рисунок 2 – Схема заполнения отдельной базы данных с параметрами CVE для дальнейших расчетов

Данные для заполнения карточки уязвимости в базе данных запрашиваются из файла с набором CVE, где у каждой CVE есть вектор относительно каждого калькулятора, для калькуляторов CVSS метрики будут рассчитываться именно на основе него, как и для OWASP, данные калькулятора EPSS и проверка находятся ли уязвимость в базе CISA CEV загружаются автоматически, по принципу нахождения соответствия. То есть из базы EPSS загрузится метрика, для нужной уязвимости, а коэффициент о нахождении уязвимости из нашей базы в базе CISA будет присвоен при нахождении соответствия в ней текущей уязвимости.

Метрики не всегда заполняются на прямую из файлов с уязвимостями из ранее описанных источников, поэтому необходимо описать как будет рассчитываться метрика для уязвимости согласно калькуляторам из рисунка 3.

Алгоритм расчета оценки Base Score по калькулятору CVSS2 v2, формулы взяты с портала БДУ ФСТЭК [32]:

1) Запуск метода calculate_base_score () класса CVSS2Calculator (код представлен в приложении);

2) Проверка заполнения всех метрик вектора согласно второй колонке из таблицы 1;

3) Замена значений метрик на коэффициенты, согласно таблице 1, например, если AV = N, то коэффициент 1.0, и так далее для остальных;

4) Расчет значения Impact, согласно формуле 1:

$$\text{Impact} = 10.41 \times (1 - (1 - C) \times (1 - I) \times (1 - A)), \quad (1)$$

где C, I, A – коэффициенты соответствующих метрик.

5) Расчет значения Exploitability, согласно формуле 2:

$$\text{Exploitability} = 20 \times AV \times AC \times Au, \quad (2)$$

Где AV, AC, Au – коэффициенты соответствующих метрик.

6) Расчет значения Base Score, согласно формуле 3:

$$BaseScore = (0.6 \times Impact + 0.4 \times Exploitability - 1.5) \times 1.176, \quad (3)$$

7) Возврат результата, возвращается рассчитанное значение Base Score и записывается в поля базы данных Impact, Exploitability, Base Score, согласно используемому калькулятору.

Таблица 1 – значение коэффициентов для метрик вектора CVSS2

Метрика	Значение	Коэффициент
AV (Access Vector)	L, A, N	0.395, 0.646, 1.0
AC (Access Complexity)	H, M, L	0.35, 0.61, 0.71
Au (Authentication)	M, S, N	0.45, 0.56, 0.704
C (Confidentiality Impact)	N, P, C	0.0, 0.275, 0.660
I (Integrity Impact)	N, P, C	0.0, 0.275, 0.660
A (Availability Impact)	N, P, C	0.0, 0.275, 0.660

Блок схема расчета Base Score по ранее упомянутому калькулятору представлена на рисунке 3.

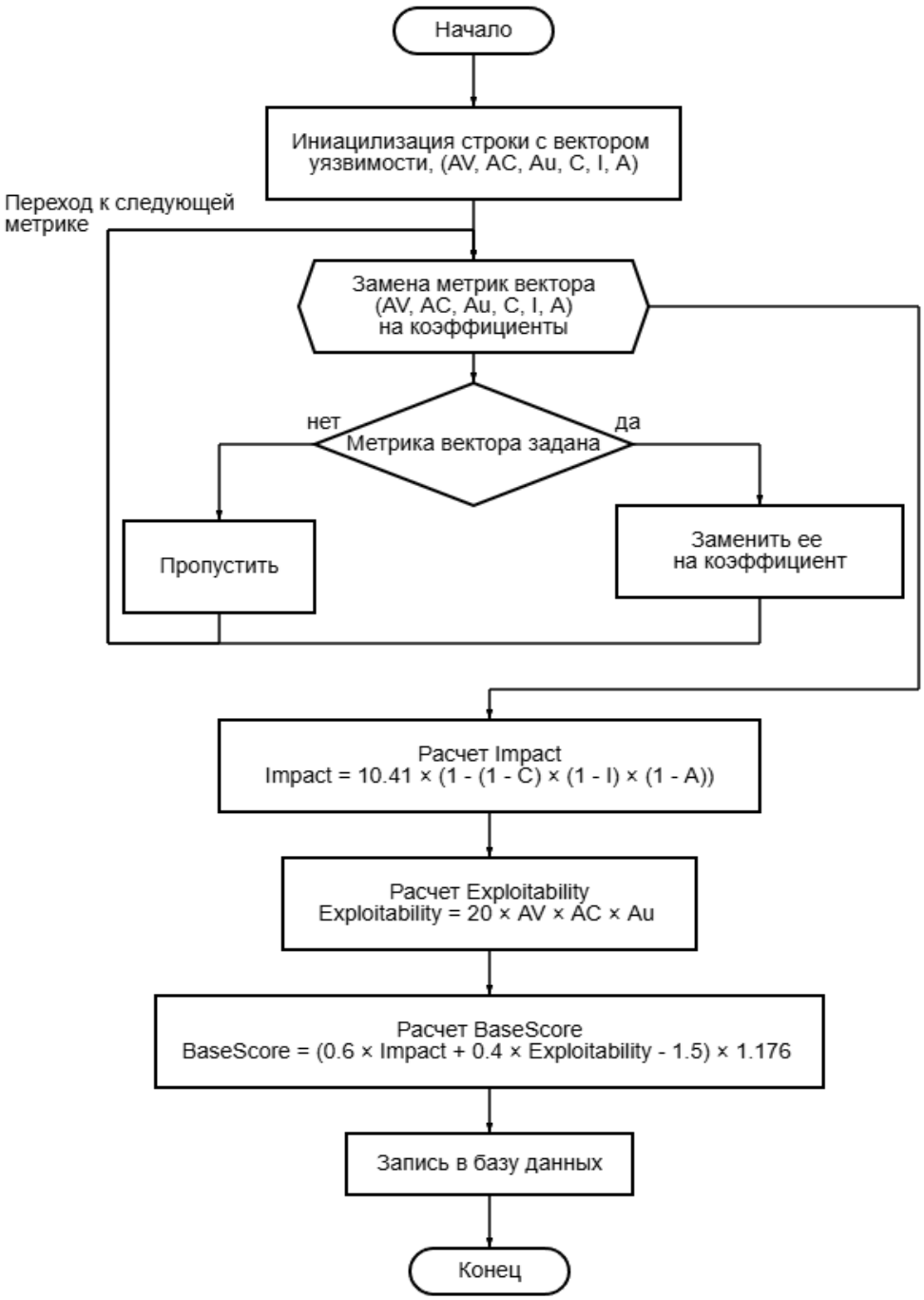


Рисунок 3 – Блок схема расчета Impact, Exploitability, Base Score по вектору уязвимости через калькулятор CVSS v2

Алгоритм расчета оценки Base Score по калькулятору CVSS v3.1 формулы взяты с портала БДУ ФСТЭК [33]:

1) Запускается метод calculate_base_score() объекта класса CVSS31Calculator (код представлен в приложении);

2) Проверка заполнения метрик вектора согласно таблице 2;

3) Замена значений метрик на коэффициенты, согласно таблице 2;

4) Расчет Расчёт ISS (Impact Sub-Score) по формуле 4:

$$ISS = 1 - (1 - C) \times (1 - I) \times (1 - A), \quad (4)$$

Где C, I, A — коэффициенты соответствующих метрик.

5) Расчёт Impact (в зависимости от Scope), Если Scope (S) = U (Unchanged), то расчет ведется по формуле 5:

$$Impact = 6.42 \times ISS, \quad (5)$$

Если Scope = C (Changed), то по формуле 6:

$$Impact = 7.52 \times (ISS - 0.029) - 3.25 \times (ISS - 0.02)^{15}, \quad (6)$$

6) Расчёт Exploitability формуле 7:

$$Exploitability = 8.22 \times AV \times AC \times PR \times UI, \quad (7)$$

Все коэффициенты берутся из таблицы 2, где коэффициент PR зависит от Scope.

7) Расчёт итогового Base Score, в случае если Impact ≤ 0, то значение будет равно 0, если Scope = U (Unchanged), то значение рассчитывается по формуле 8:

$$Base\ Score = Impact + Exploitability, \quad (8)$$

Если Scope = C (Changed), то значение будет рассчитываться по формуле 9:

$$Base\ Score = 1.08 \times (Impact + Exploitability), \quad (9)$$

8) Возврат значения Base Score и записывается в поля базы данных значения Impact, Exploitability, Base Score, согласно используемому калькулятору.

Таблица 2 – значение коэффициентов для метрик вектора CVSS v3.1

Метрика	Значение	Коэффициент
AV (Attack Vector)	N, A, L, P	0.85, 0.62, 0.55, 0.20
AC (Attack Complexity)	L, H	0.77, 0.44
PR (Privileges Required)	N, L, H	U (Unchanged): 0.85, 0.62, 0.27; C (Changed): 0.85, 0.68, 0.50
UI (User Interaction)	N, R	0.85, 0.62
C (Confidentiality Impact)	N, L, H	0, 0.22, 0,56
I (Integrity Impact)	N, L, H	0, 0.22, 0,56
A (Availability Impact)	N, L, H	0, 0.22, 0,56

Блок схема расчета Base Score по ранее упомянутому калькулятору представлена на рисунке 4.

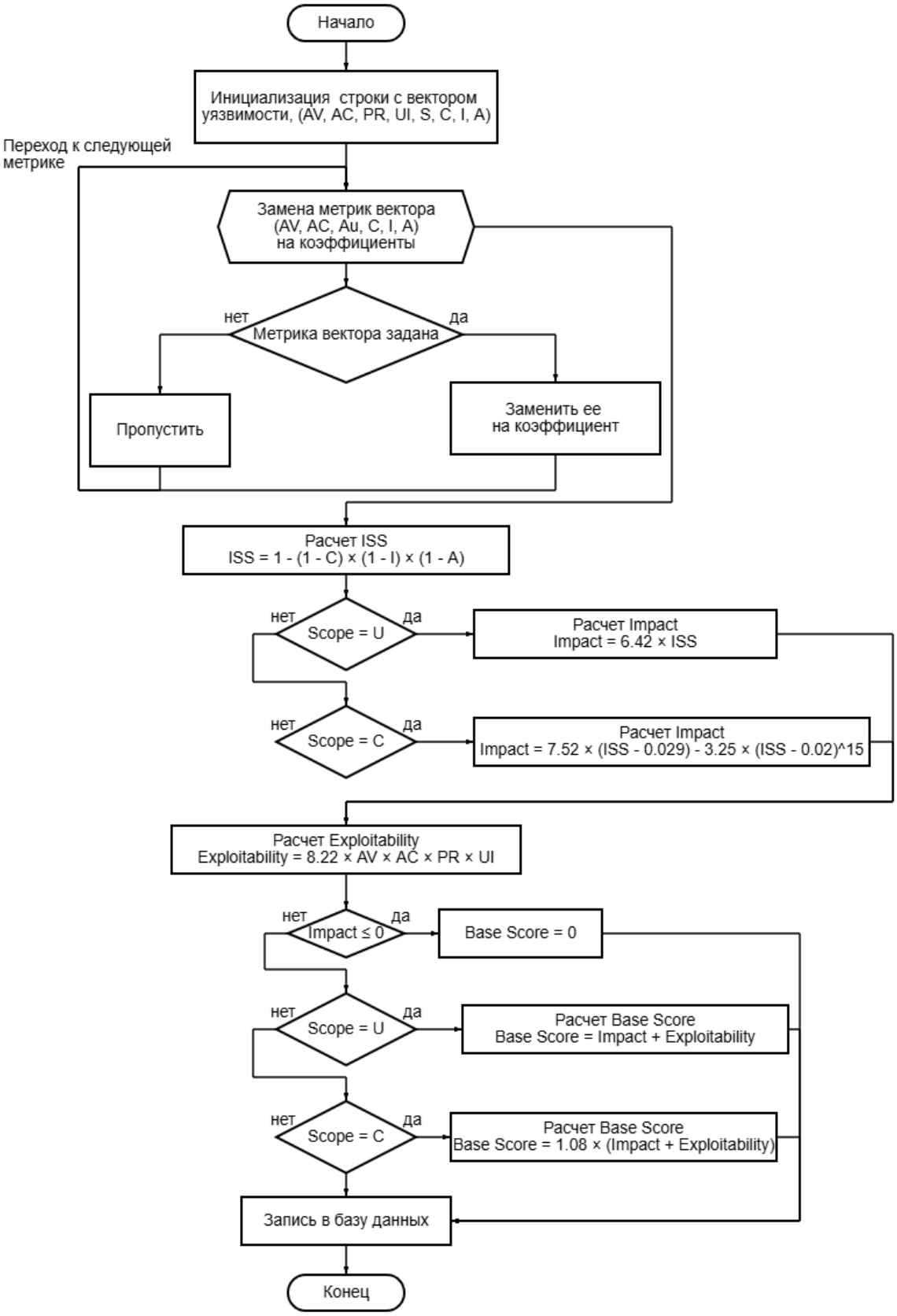


Рисунок 4 - Блок схема расчета Impact, Exploitability, Base Score по вектору через калькулятор CVSS v3.1

Алгоритм расчета оценки Base Score по калькулятору CVSS v4 формулы взяты с портала БДУ ФСТЭК [34]:

1) Запускается метод calculate_base_score() объекта класса CVSS4Calculator (код представлен в приложении);
2) Проверка заполнения метрик вектора согласно таблице 3;
3) Расчет Exploitability по формуле 10:

$$\text{Exploitability} = 8.22 \times AV \times AC \times AT \times PR \times UI, \quad (10)$$

4) Расчет vs_impact согласно формуле 11:

$$\text{vs_impact} = 1 - (1 - VC) \times (1 - VI) \times (1 - VA), \quad (11)$$

5) Расчет ss_impact согласно формуле 12:

$$\text{ss_impact} = 1 - (1 - SC) \times (1 - SI) \times (1 - SA), \quad (12)$$

6) Расчет Impact по формуле 13:

$$\text{Impact} = 6.42 \times \text{vs_impact} + 3.21 \times \text{ss_impact}, \quad (13)$$

7) Расчет BaseScore по формуле:

$$\text{BaseScore} = \sqrt{(\text{Impact}^2 + \text{Exploitability}^2)}, \quad (14)$$

Обозначения 'AV', 'AC', 'AT', 'PR', 'UI', 'VC', 'VI', 'VA', 'SC', 'SI', 'SA' это табличные значения, взятые из таблицы 3 [34].

Таблица 3 – значение коэффициентов для метрик вектора CVSS v3.1

Метрика	Возможные значения	Коэффициенты
AV (Access Vector)	N, A, L, P	0.85, 0.62, 0.55, 0.20
AC (Attack Complexity)	L, H	0.77, 0.44
AT (Attack Techniques)	N, P	0.85, 0.62
PR (Privileges Required)	N, L, H	0.85, 0.62, 0.27
UI (User Interaction)	N, P, A	0.85, 0.62, 0.45
VC (Vulnerable System CI)	H, L, N	0.56, 0.22, 0.00
VI (Vulnerable System II)	H, L, N	0.56, 0.22, 0.00
VA (Vulnerable System AI)	H, L, N	0.56, 0.22, 0.00
SC (Subsequent System CI)	H, L, N	0.56, 0.22, 0.00
SI (Subsequent System II)	H, L, N	0.56, 0.22, 0.00
SA (Subsequent System AI)	H, L, N	0.56, 0.22, 0.00

Блок схема расчета Base Score по калькулятору CVSS v4 представлена на рисунке 5.

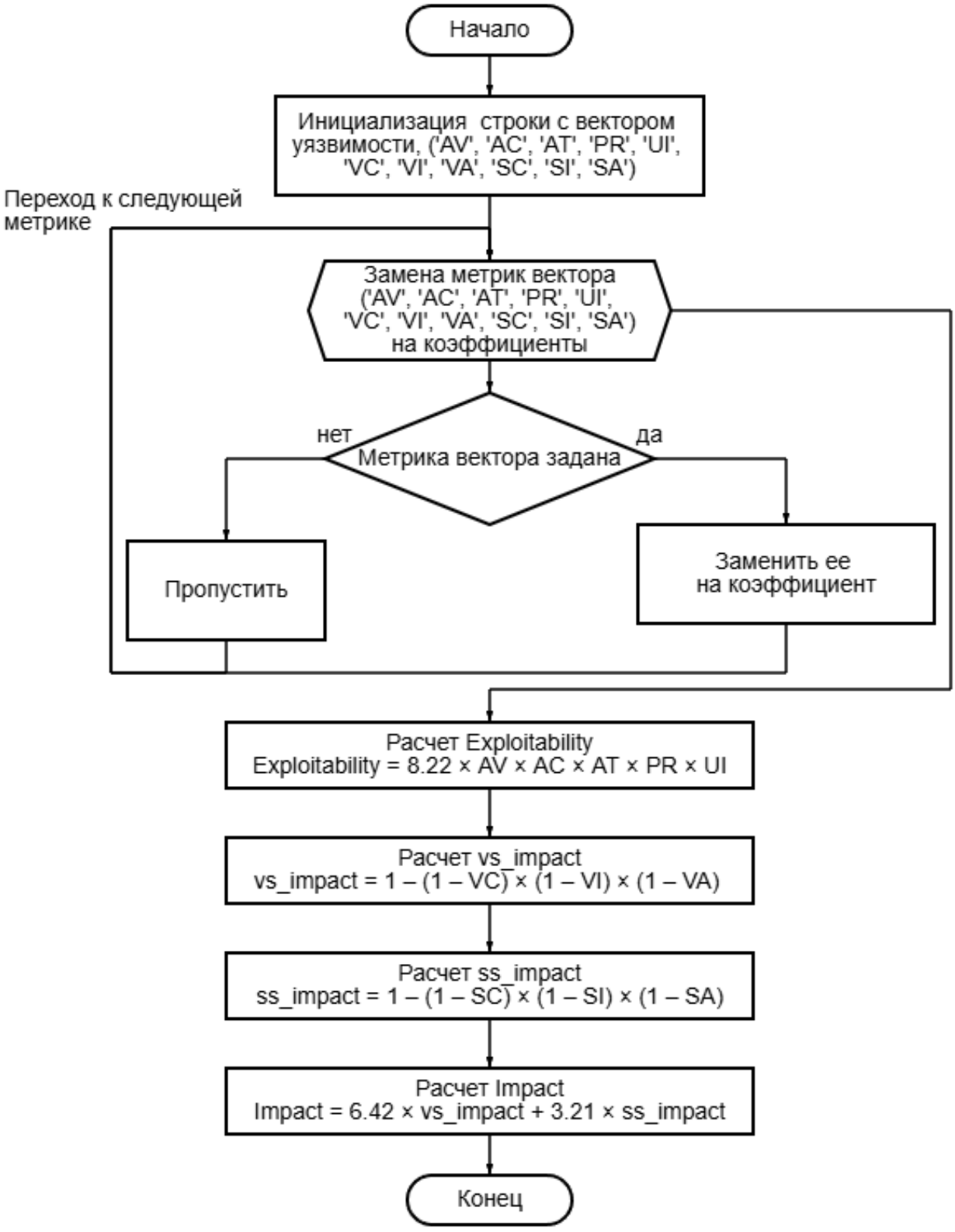


Рисунок 5 - Блок схема расчета Impact, Exploitability, Base Score по вектору через калькулятор CVSS v4

Разберем теперь метрики вектора OWASP, они делятся на 4 группы факторов, формулы взяты с официального источника OWASP, разработовавшего калькулятор [35]:

Факторы угрозы:

- Skill Level (SL);
- Motive (M);
- Opportunity (O);
- Size (S).

Факторы уязвимости:

- Ease of Discovery (ED);
- Ease of Exploit (EE);
- Awareness (A);
- Intrusion Detection (ID).

Факторы технического воздействия:

- Loss of Confidentiality (LC);
- Loss of Integrity (LI);
- Loss of Availability (LAV);
- Loss of Accountability (LAC).

Факторы бизнес-воздействия:

- Financial Damage (FD);
- Reputation Damage (RD);
- Non-compliance (NC);
- Privacy Violation (PV).

Уточнение, далее приведенный алгоритм расчета метрик уязвимости через калькулятор OWASP является приведением вектора CVSS v3.1 в вектор OWASP, способ был взят с ранее упомянутого официального сайта калькулятора [35]:

1) Проверка полноты (наличия вектора), если вектор не полный, то согласно таблице 3 заменить метрики вектора на коэффициенты;

2) Вычисление Impact согласно формуле 15:

$$Impact = 10.41 \times (1 - (1 - C)(1 - I)(1 - A)), \quad (15)$$

3) Вычисление Exploitability согласно формуле 16:

$$Exploitability = 20 \times AV \times AC \times Au, \quad (16)$$

4) Вычисление Base Score согласно формуле 17:

$$Base\ Score = (0.6 \times Impact + 0.4 \times Exploitability - 1.5) \times 1.176, \quad (17)$$

5) Запись Base Score в базу данных

Таблица 3 - значение коэффициентов для метрик вектора OWASP

OWASP-фактор	Источник из CVSS	Условие присвоения	Значение
Skill Level (sl)	CVSS Base Score	Если Base Score ≥ 7	7
		Если Base Score ≤ 7	4
Motive (m)	—	По умолчанию	6
Opportunity (o)	CVSS Base Score	Если Base Score ≥ 9	8
		Если Base Score ≤ 9	5
Size (s)	—	По умолчанию	5
Ease of Discovery (ed)	CVSS Base Score	Если Base Score ≥ 7	8
		Если Base Score ≤ 7	5
Ease of Exploit (ee)	CVSS Base Score	Если Base Score ≥ 6	7
		Если Base Score ≤ 6	5
Awareness (a)	—	По умолчанию	5
Intrusion Detection (id)	—	По умолчанию	3
Loss of Confidentiality (lc)	confidentialityImpact	confidentialityImpact × 8	0.0 – 8.0
Loss of Integrity (li)	integrityImpact	integrityImpact × 8	0.0 – 8.0

Продолжение таблицы 3

OWASP-фактор	Источник из CVSS	Условие присвоения	Значение
Loss of Availability (lav)	availabilityImpact	availabilityImpact × 8	0.0 – 8.0
Loss of Accountability (lac)	—	По умолчанию	3
Financial Damage (fd)	lav	min(9, lav × 1.5)	0.0 – 9.0
Reputation Damage (rd)	li	min(9, li × 1.5)	0.0 – 9.0
Non-compliance (nc)	—	По умолчанию	3
Privacy Violation (pv)	lc	min(9, lc × 1.5)	0.0 – 9.0

Блок схема алгоритма расчета метрик по калькулятору OWASP представлена на рисунке 6.

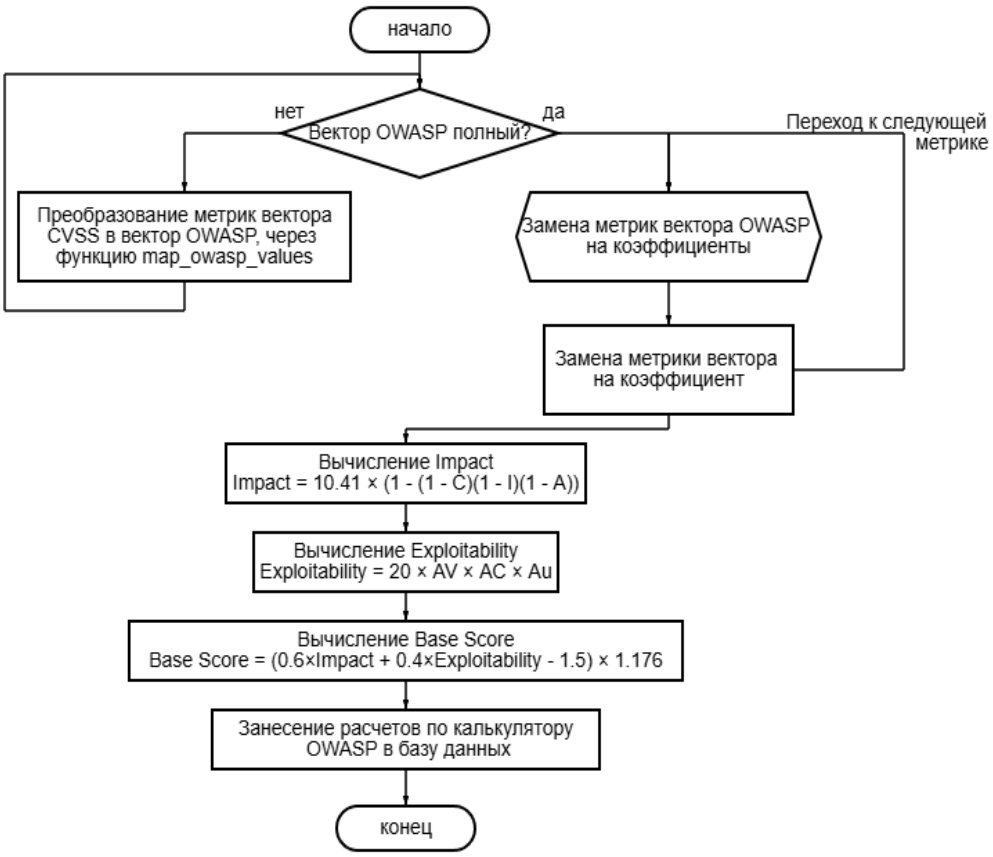


Рисунок 6 – Блок схема расчета Impact, Exploitability, Base Score по вектору через калькулятор OWASP

Калькулятор CVSS был впервые создан достаточно давно, в силу чего потерпел много изменений, а именно представление его в нескольких версиях. Самыми популярными являются версии CVSS 2, 3.1 и 4, однако из всех этих версий на вооружении была взята вторая версия. Вторая версия была создана еще в 2007 году и является прародителем всех последующих, в силу этого она не учитывает большинство современных параметров и ее метрики неточны в нынешних реалиях. Четвертая версия относительно свежая, она была выпущена в 2023 году, она учитывает большинство современных аспектов в области кибербезопасности, однако еще не успела получить широкое распространение, в силу этого и была выбрана версия CVSS 3.1, которая получила широкое использование за время своего существования [32-34]. Несмотря на то, что расчеты будут вестись по ранее упомянутой версии, метрики по оставшимся не стоит исключать из базы данных, так как они могут пригодиться для дальнейших исследований и позволят оставить ребро гибкости в звене развития разработанного в ходе работы программного обеспечения. Помимо этого, в базу будут добавлены метрики, рассчитанные с помощью калькуляторов EPSS, он в дальнейшем нам пригодится и OWASP. Все эти параметры в сборе составляют уникальную целостную базу данных, не имеющую аналогов. Для удобства пользования разработанной базой данных был создан инструмент, позволяющий пользователю легко с ней взаимодействовать и получать отчеты в удобном для него формате, пример его работы будет представлен в приложении [32-34].

Вернемся к метрикам CVSS, которые мы решили брать для дальнейших расчетов. Проблема данного калькулятора состоит в том, что он не учитывает влияние уязвимости на все три компонента безопасности, а именно: целостность, доступность и конфиденциальность. В нем присутствуют поля, описывающие влияние на эти три параметра, но они используются для итоговой оценки уязвимости, обобщенной, нас такой исход не устраивает, поэтому необходимо найти решение и описать как уязвимость влияет на каждый компонент безопасности. Для понятного представления можно прибегнуть к кругам Эйлера на рисунке 7.

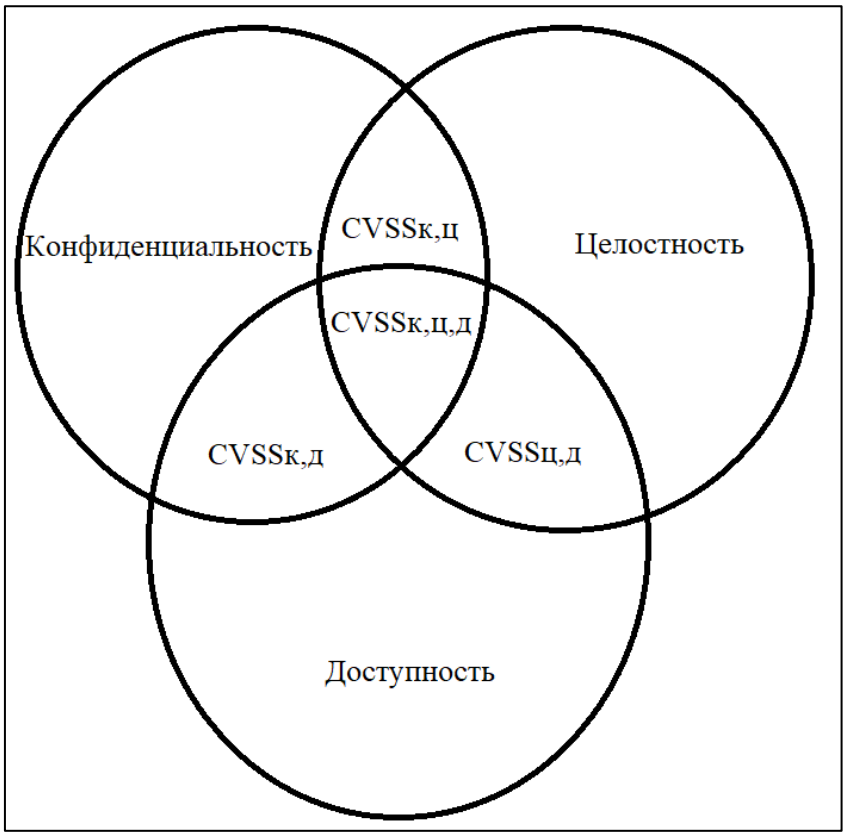


Рисунок 7 – Диаграмма пересечений базовых метрик безопасности в CVSS: конфиденциальность, целостность и доступность

Диаграмма описывает то, как будет выглядеть итоговая оценка в случае влияния уязвимости на какой-либо из компонентов.

Наша задача состоит теперь в нахождении весов, которые опишут влияние уязвимости на конкретное свойство безопасности. В этой задаче стоит использовать метод Шепли из теории кооперативных игр. Этот метод позволяет распределить совокупный результат (в нашем случае — Base Score CVSS) между участниками (в нашем случае — метриками К, Ц, Д) на основе их маргинального вклада во всех возможных комбинациях.

Пусть:

N = {К,Ц,Д} — множество метрик (игроков);

v(S) — функция, вычисляющая Base Score при наличии подмножества метрик S⊆N;

$\phi_i(v)$ — значение Шепли для метрики i∈N, отражающее её вклад в итоговую оценку.

Тогда справедливо что:

$$\phi_i(v) = \frac{1}{N!}\sum_p [v(S_p(i) \cup \{i\}) - v(S_p(i))], \quad (18)$$

где p $\in$ число перестановок N

$\phi_i(v)$ - вклад метрики $i \in$ {К, Ц, Д} в CVSS Base Score;

$S_p(i)$ - множество метрик, добавленных до i в перестановке p;

$v(S)$ - функция расчета Base SCore при наличии только метрик из S.

$v(S_p(i) \cup \{i\}) - v(S_p(i))$ — маргинальный вклад метрики i в конкретной перестановке.

Для трёх метрик (К, Ц, Д) существует 6 перестановок. Для каждой из них оценивается прирост Base Score при добавлении метрики i к предыдущим. Итоговое значение Шепли — это среднее по всем перестановкам.

В результате вычисления для К, Ц, Д мы можем найти их веса:

$$w_{\text{К,Ц,Д}} = \frac{\phi_{\text{к,ц,д}}}{\phi_{\text{кцд}}} = \frac{\phi_{\text{к,ц,д}}}{\phi_{\text{к}}+\phi_{\text{ц}}+\phi_{\text{д}}}, \quad (19)$$

Отсюда следует формула для вычисления CVSS под каждый компонент системы:

$$CVSS_{\text{К,Ц,Д}} = CVSS_{Base\ Score} \times w_{\text{К,Ц,Д}}, \quad (20)$$

На основе этой формулы можно составить формулу нормированного ущерба, но перед этим расширим нашу базу данных уязвимостями находящимися в CISA KEV, этот источник содержит статистику об уязвимостях, которые используются злоумышленниками повсеместно, тем самым он указывает на то, что уязвимости должен быть уделено особое внимание. Отсюда получаем нормированный ущерб:

$$\underline{U_{\text{К,Ц,Д}}} = CVSS_{\text{К,Ц,Д}} \times K_{CISA\ KEV} \times \frac{1}{35}, \quad (21)$$

где К, Ц, Д – это влияние на конфиденциальность, целостность и доступность информации;

$K_{CISA\ KEV}$ - коэффициент поправки популярности эксплуатации уязвимости согласно ресурсу CISA KEV, значение которого равно 1, если уязвимость не находится в базе CISA и 3,5 - если уязвимость находится в базе данных;

$\frac{1}{35}$ – коэффициент нормализации, приводящий значение ущерба в предел от 0 до 1, значение нижней части дроби 35 основано на максимальном значении $CVSS_{\text{К,Ц,Д}}$, равное 10 и максимальном значении $K_{CISA\ KEV}$ равное 3,5.

Число равное 3,5 в качестве коэффициента было выбрано исходя из статистической информации от Bitsight, говорящей о том, что уязвимости, находящиеся в этой базе, несут в себе большую опасность и в 60% случаев устраняются по истечении отведенных на это сроков [46]. Число 3.5 вышло из деления 621 на 174, так как именно столько устраняются критичные и некритичные уязвимости согласно отчету ранее упомянутой компании [46].

Формула нормированного риска выглядит следующим образом:

$$\underline{Risk} = P \times \underline{U}, \quad (22)$$

где Р - вероятность реализации угрозы через уязвимость,

$\underline{U}$ - нормированный ущерб активу в результате реализации угрозы

В классической формуле риска присутствует переменная равная стоимости атакуемой системы, мы имеем право исключить эту переменную, так как нами рассматривается атака в рамках одного ресурса, а не целой системы.

Из формул 16 и 17 вытекает:

$$\underline{Risk_{CVE}^{\text{К,Ц,Д}}} = EPSS * \underline{U_{\text{К,Ц,Д}}} = \frac{EPSS \times CVSS_{\text{К,Ц,Д}} \times K_{CISA\ KEV}}{35}, \quad (23)$$

где EPSS – значение калькулятора EPSS по эксплуатируемой уязвимости, которое заранее было добавлено в базу данных.

На основе формул 18-23 был разработан блок расчета риска для атак эксплуатирующую единичную уязвимость. Перейдем к разбору алгоритма этого блока.

Так как в базе данных CVSS метрики, отвечающие за конфиденциальность, целостность и доступность представлены не в числовом, а в буквенном формате, то нужно будет сделать их перевод согласно официальному источнику калькулятора, данные для перевода представлены в таблице 4 [33].

Таблица 4 – коэффициенты К, Ц, Д пропорционально CVSS [33].

Свойство информации	Обозначение в CVSS	Коэффициент
Конфиденциальность (К) Confidentiality Impact	C	N (None) = 0 L (Low) = 0,22 H (High) = 0,56
Целостность (Ц) Integrity Impact	I	
Доступность (Д) Availability Impact	A	

Алгоритм вычисления риска для единичных атак содержит следующие этапы:

1) Выбор техники пользователем;
2) Считывание из базы данных всех уязвимостей связанных с этой техникой;
3) Считывание метрик необходимых для расчета;
4) Расчет весов Шепли на основе считанных метрик;
5) Расчет риска для трех параметров, с учетом ранее посчитанных весов и расчет суммарного риска;
6) Вывод результатов расчета.

Так же алгоритм представлен в виде блок схемы, описанной на рисунке 8, формулы расчетов были представлены ранее, это формулы 18-23.



Рисунок 9 – Алгоритм расчета для единичных атак

2.2.2 Алгоритм расчета риска для множественных атак

Вводя параметр C_i как ценность защищаемого ресурса i-ой компоненты, функция риска принимает следующий вид (24):

$$Risk(k, C_i, n, p) = k \cdot C_i \cdot C_n^k p^k (1-p)^{n-k}, \quad (24)$$

С учетом вводимых параметров ценности и вероятности успешной атаки, управление риском осуществляется с помощью квантильного метода перенормировки вероятностей, что позволит достичь условия равенства максимумов (25) для каждого слагаемого в сумме (26):

$$max\{P_1 k_1 C_1\} = max\{P_2 k_2 C_2\} = max\{P_3 k_3 C_3\}, \quad (25)$$

$$Risk_{\Sigma} = Risk_1 + Risk_2 + Risk_3, \quad (26)$$

Управление представляет собой несколько этапов: перенормировка самих вероятностей, перерасчет рисков и выравнивание их с целью минимизации значений, удовлетворяющих условию (26). Для описания работы с множеством значений рисков и соответствующих им вероятностей для каждой уязвимости, удобно представлять эти величины через набор векторов, по которым можно ввести и осуществить операцию поиска максимумов и минимумов соответственно (27):

$$\max_k Risk_i(k), \quad (27)$$

где через данное соотношение обозначается максимальная величина вычисленного риска для выбранной уязвимости (максимум из всех значений рисков для одной уязвимости).

Для заданного квантиля q необходимо вычислить число успешных атак $k_q^{(i)}$, которое покрывает q-тую долю биномиального распределения. Сделать это можно с помощью обратной функции распределения (28):

$$k_q^{(i)} = F^{-1}(q; n, p_i), \tag{28}$$

где $\mathrm{F}^{-1}(\mathrm{q};\mathrm{n},\mathrm{p_i})$ – обратная функция биномиального распределения,

$\mathrm{k_q^{(i)}}$ – количество успешных атак (будет достигнуто или превышено с вероятностью q для i-й уязвимости);

q – целевой квантиль.

Позиция целевого квантиля для i-й уязвимости $T(p_i)$ определяется как:

$$T(p_i) = c - \frac{w}{2} + w \cdot \frac{p_i - p_{min}}{p_{max} - p_{min}}, \tag{29}$$

где c – центр целевого интервала;

w – ширина целевого интервала;

$\mathrm{p_{min}}$ – минимальная исходная вероятность из списка всех уязвимостей;

$\mathrm{p_{max}}$ – максимальная исходная вероятность из списка всех уязвимостей.

На целевую позицию следует наложить условие ограничения значений вероятности, которое можно описать функцией $\theta(\mathrm{x})$, во избежание получения недопустимых значений (30):

$$\theta(\mathrm{x}) = \max(0{,}05, \min(0{,}95, x)), \tag{30}$$

Поскольку функция поиска целевой позиции и количества успешных атак ($T(p_i)$ и $k_q^{(i)}$) монотонны, то справедливо заметить сохранение следующего неравенства (31) при выполнении операции перенормировки:

$$p_i < p_j \Longrightarrow p_i^* < p_j^*, \tag{31}$$

где p^* – перенормированное значение исходной вероятности с указанными параметрами.

Таким образом, соотношение для перераспределения по квантильному методу перенормировки (32) имеет вид:

$$p^* = T(p) \cdot \frac{F^{-1}(q; n, p)}{n \cdot q}, \tag{32}$$

Далее следует рассчитать значение рисков для перенормированных значений вероятности, после чего скорректировать их с целью достижения оптимального значения. Для описания скорректированного риска введем величину $Risk^*$ – результат воздействия некоторого отображения $\varphi : R \to R$, позволяющего достичь условия максимума (26). Поскольку суммарный оптимальный риск равен сумме оптимальных, достаточно рассмотреть методику расчета $Risk^*$ для одного слагаемого, аналогичным образом применяя ее для каждого последующего для заданного набора исходных рисков.

Поскольку коррекция происходит в каждой точке распределения, то для описания отображения удобно представить риск в виде зависимости от k, опуская прочие параметры – отображение позволяет сохранить форму распределения при изменении начальных величин. Таким образом, φ представимо как (33):

$$\varphi\big(Risk(k)\big) = Risk(k)^* = Risk(k) \cdot \varepsilon\,, \tag{33}$$

где $Risk^*(k)$ – перенормированное значение риска,

$Risk(k)$ – значение начального риска для k успешных атак,

ε – некоторый коэффициент коррекции i-того значения.

Также справедливо заметить, что для данное отображение в контексте рисков, сохраняет аргументы, при которых достигается максимум (34):

$$argmax_k\ Risk(k) = argmax_k\ Risk^*(k), \tag{34}$$

Для достижения управления введем функционал (35), выделяющий как метрику квадрат отклонений и ограничивающий чрезмерное отклонение корректирующих коэффициентов:

$$J(\varepsilon) = \sum_{i=1}^{m}\sum_{j=1}^{m}\left(\varepsilon_i \cdot \max_k Risk_i(k) - \ \varepsilon_j \cdot \max_k Risk_j(k)\right)^2 + \ \lambda \sum_{i=1}^{m}(\ln \varepsilon)^2\ , \tag{35}$$

Оптимальное управление риском достигается, если коэффициент коррекции обратно пропорционален пиковому значению (36), так как при таком значении коэффициента первое слагаемое функционала (35) обнуляется:

$$\varepsilon_i = \frac{\max\left(Risk(k)\right)}{\max\limits_k Risk_i(k)}\ , \tag{36}$$

Таким образом достигается управление риском для множественных атак, позволяя, изменяя параметры для квантильной перенормировки вероятности, достичь оптимального значения суммарного риска, попадающего в ограниченный диапазон.

Математическая модель, предложенная для расчета и управления рисками множественных атак, реализуется с помощью последовательного алгоритма, включающего в себя чтение, обработку данных и визуализацию полученных зависимостей на графиках. Общий вид алгоритма, используемого для проведения расчетов, изображен на рисунке 10:



Рисунок 10 – Блок-схема общего алгоритма обработки данных

Общий алгоритм программы включает в себя ввод набора уязвимостей, для которых производятся вычисления, а также выбор вида расчета, в зависимости от вида атаки. Каждый соответственный расчет вынесен в отдельную блок-схему для упрощения чтения и повышения модульности оформления алгоритма. Расчет для начального распределения рисков, вычисленных для введенных данных об уязвимостях, изображен на рисунке 11.



Рисунок 11 – Блок-схема алгоритма расчета начальных рисков без управления

Начальный расчет рисков включает в себя несколько последовательных операций с циклом, позволяющих получить вектор соответствующих значений для введенных данных, индекс которых совпадает. Для каждой уязвимости вычисляется комбинаторная формула и умножение на ценность, после чего результат операции передается в блок построения графиков. Графики отображаются несколькими линиями на одном «теле», позволяя визуализировать отличия рассчитанных величин

и наглядно продемонстрировать начальные распределения без применения алгоритма управления. Блок-схема, описывающая алгоритм квантильной перенормировки и управления риском изображена на рисунке 12:



Рисунок 12 – Блок-схема алгоритма перенормировки рисков и управления ими

Управление возможно тогда и только тогда, когда был произведен начальный расчет: данные о рассчитанных рисках по возможности переиспользуются, позволяя сэкономить вычислительные ресурсы и перестроить график сразу по действию пользователя, предоставляя наглядную разницу между результатом применения алгоритма и начальным расчетом.

Помимо данных о начальных уязвимостях, алгоритм управления включает в себя ввод данных с параметрами квантильного перераспределения: целевого квантиля, центра и ширины интервала, на котором происходит перенормировка. Алгоритм также включает в себя множество последовательных операций с несколькими циклами, тело которых реализует перенормировку вероятностей, заключающуюся в вычислении соответствующих значений мат. модели – обратной функции биномиального распределения вероятностей и свойств целевого интервала –, а также нахождение оптимального управления получившихся функций риска, основанных на метрике из функционала. После вычисления значений для каждой уязвимости, алгоритм вычисляет суммарный риск как сумму оптимизированных рисков и передает вектор всех значений в модуль построения графиков. Графики выводятся по завершению всех операций аналогично, на одном «теле», и позволяют при изменении параметров интерактивно подбирать необходимые, позволяющие ограничивать суммарный риск в нужном интервале для выбора оптимальной стратегии защиты. Модуль построения графиков предусматривает функцию поиска нужных экстремальных точек функции суммы рисков, что позволяет графически отобразить искомый диапазон. Вышеописанный алгоритм является уникальным и ранее не рассматривался в сфере информационной безопасности, что придает работе теоретической значимости.

2.2.3 Алгоритм расчета риска для многовекторных атак

В этой главе будет рассмотрен способ реализации, или алгоритм, анализа и расчета многовекторных атак.

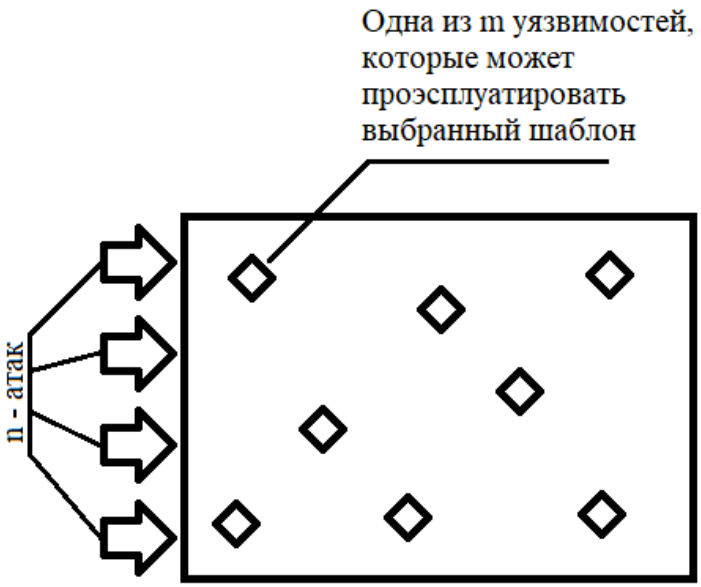


Рисунок 13 - Система из m уязвимостей под воздействием n атак

В случае многовекторных атак (рисунок 13) для исследования уязвимостей и рисков применимо мультиномиальное распределение:

$$P(x_1, \dots, x_m, n, p_1, \dots, p_m) = \frac{\mathrm{n!}}{x_1!,\dots, x_m!}\left(p_1^{\mathrm{x_1}}, \dots, p_{\mathrm{m}}^{\mathrm{x_m}}\right), \quad (37)$$

где n – общее число атак;

$x_i = 0, 1, \dots,$ где $x \in Z, i = 1(1)m;$

$$\sum_{i=1}^{m} x_i = n;$$

$$\sum_{i=1}^{m} p_i = 1;$$

$p_i -$ вероятность успешной эксплуатации $i -$ ой уязвимости;

$np_i -$ матожидание количества успешных атак на $i -$ ую уязвимость.

Блок-схема алгоритма работы модуля расчета многовекторных атак представлена на рисунке 14.



Рисунок 14 – Блок-схема алгоритма работы модуля расчета многовекторных атак

Реализация в разрабатываемом программно-техническом комплексе по алгоритму рисунка 14 немного отличается ввиду особенностей системы. Во избежание переполнений при работе с большими степенями и значениями факториалов использовались логарифмические функции.

Логарифм факториала n:

$$\log(n!) = \Gamma(n+1), \tag{38}$$

где Γ – гамма-функция, обобщающая понятие факториала на комплексные и вещественные числа.

Сумма логарифмов факториалов x_i:

$$\sum \log(x_i!), \tag{39}$$

Сумма $x_i \log(p_i)$:

$$\sum x_i \log(p_i), \tag{40}$$

Итоговое значение вероятности вычисляется следующим образом:

$$\mathrm{P} = e^{\log(n!) - \sum \log(x_i!) + \sum x_i \log(p_i)}, \tag{41}$$

где e – экспоненциальная функция

Следующий шаг – расчет риска:

$$Risk = \left(\sum_{i=1}^{m} x_i \, u_i \right) P(x_1, \dots, x_m), \tag{42}$$

где $u_i = \alpha C_i$ – ущерб от одной успешной атаки,

α – коэффициент потери ценности

C_i – ценность ресурса

Для реализации алгоритма работы использовались следующие формулы:

Нормировка вероятностей:

$$p_i^* = \frac{p_i}{\sum p_i}, \tag{43}$$

где p_i^* - нормированное значение вероятности.

Расчет ожидаемых успешных атак:

$$x_i \approx np_i^*, \tag{44}$$

Расчет ущерба:

$$u_i = \alpha C_i, \tag{45}$$

Суммарный ущерб для исхода x:

$$U = \sum_{i=1}^{m} x_i u_i, \tag{46}$$

где U – суммарный ущерб.

Далее следует оптимизация ресурсов с целью минимизировать риск путем перенормировки C_i при фиксированной общей ценности $\sum C_i = C_\Sigma$.

Формула оптимальных C_i:

$$C_i' = C_\Sigma \frac{w_i}{\sum_{j=1}^{m} w_j}, \tag{47}$$

где $w_i = \frac{1}{np_i\alpha}$;

w_i – вес, рассчитывается как величина обратная произведению общего числа атак n, вероятности успеха атаки p_i и коэффициента потерь α. Это обеспечивает оптимальное распределение ресурсов: уязвимости с низкой вероятностью успеха получают больше ресурсов, так как их защита обеспечивает максимальное снижение потенциального ущерба. Уязвимости же с высоким p_i получают меньше ресурсов из-за низкой эффективности их защиты. Данный подход в сфере информационной безопасности ранее не наблюдался, поэтому его присутствие в работе добавляет ей новизны и теоретической значимости.

3 Проектирование программно-технического комплекса

3.1 Общее описание архитектуры комплекса

Архитектура разработанного программно-технического комплекса моделирования рисков кибератак включает в себя набор взаимосвязанных компонентов, каждый из которых отвечает за свою функцию, а именно сбор, хранение, анализ, визуализацию информации об уязвимостях и расчет количественных показателей риска. В совокупности модули позволяют моделировать различные сценарии кибератак, оценивать их риск с графическим отображением. Кроме того, имеется возможность управления рисками, что в совокупности с визуализацией дает широкое пространство для анализа кибератак.

На рисунке 15 проиллюстрирована высокоуровневая архитектура программно-технического комплекса. Далее подробно будут разобраны составляющие части главных модулей.

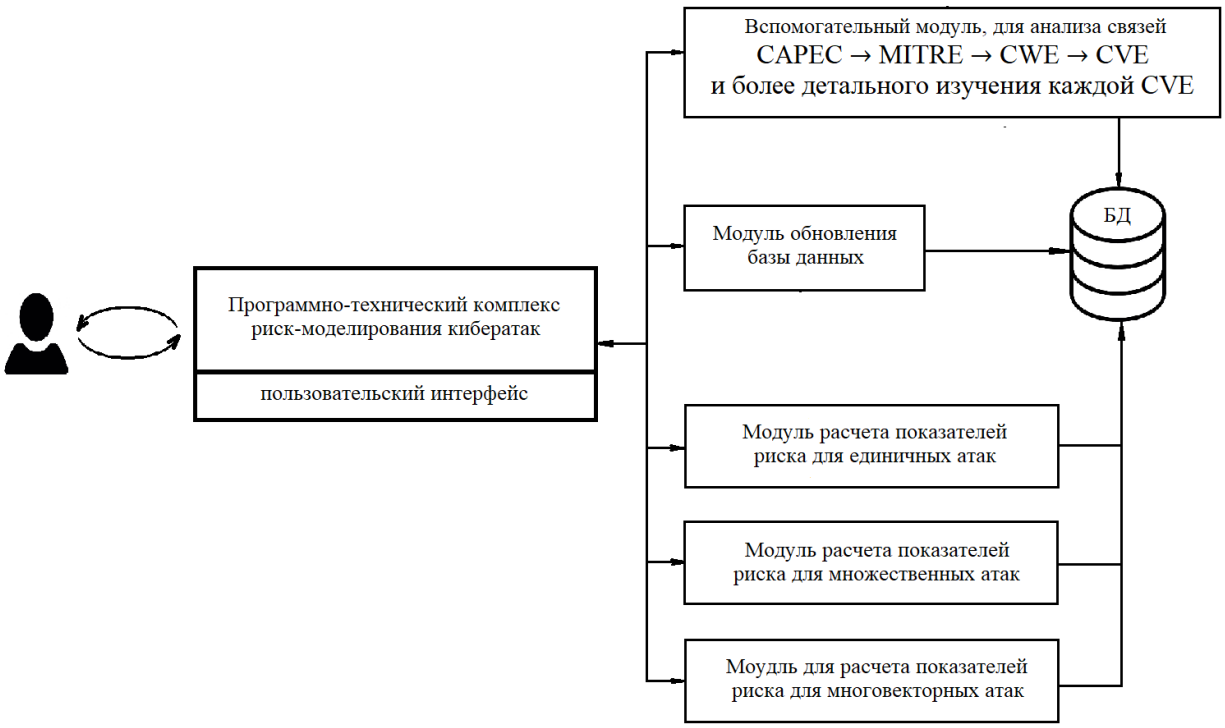


Рисунок 15 – Общая архитектура программного комплекс риск моделирования кибератак

Следуя наполнению рисунка 15, необходимыми к разработке являются следующие компоненты системы:

- база данных;
- обслуживание базы данных;
- расчет показателей риска для одиночных атак;
- расчет показателей риска для множественных атак;
- расчет показателей риска при многовекторных атаках;
- модуль анализа коммуникаций CAPEC → MITRE → CWE → CVE и детальное изучение CVE;
- пользовательский интерфейс.

Фундаментальным компонентом комплекса является база данных, хранящая в себе необходимую информацию для исследования, а именно информацию о:

- CVE;
- CWE;
- MITRE ATT&CK;
- CAPEC;
- значениях из калькуляторов для CVE, таких как CVSS (версии 2, 3, 3.1 и 4), EPSS, OWASP.

База данных выполняет функцию центрального хранилища информации и является ключевым компонентом программного обеспечения. Она обеспечивает своевременную передачу данных между модулями системы, тем самым обеспечивая корректную работу всех зависимых компонентов.

Следующий этап разработки — модуль обновления базы данных, который отвечает за поддержание информации в актуальном состоянии. Он обеспечивает долгосрочную работу системы за счет автоматической синхронизации данных с внешними источниками. В связи с необходимостью интеграции со сторонними сервисами архитектура данного модуля достаточно сложна. Подробный анализ его структуры и функционала можно найти в соответствующем разделе документации.

Далее следуют модули для расчета различных типов атак и модуль для информирования пользователя о подключениях CAPEC → MITRE → CWE → CVE и

о каждом CVE в отдельности. Модули не связаны напрямую в логике системы. Однако второй модуль расширяет функционал первого: пользователь может искать недостающие данные до или после расчета, что помогает получать более точные результаты. Однако модуль расчета индикаторов риска остается приоритетным, так как его результаты позволяют оптимизировать связанный модуль на этапе разработки. Ядром программно-технического комплекса является набор расчетных модулей:

- модуль для расчета показателей риска для одиночных атак, предназначенный для анализа риска, связанного с одиночной атакой, основанной на одной уязвимости;

- модуль расчета показателей риска для множественных атак, предназначенный для анализа повторяющихся атак, направленных на разные части системы с использованием одной или нескольких уязвимостей;

- модуль расчета показателей риска для многовекторных атак, предназначенный для анализа сложных сценариев атак, включающих несколько векторов воздействия и уязвимостей.

Разработка вспомогательного модуля, обоснованного набором основных модулей для расчета и анализа, необходима для упрощения работы с наборами уязвимостей, демонстрируя их связи в рамках выбранного CAPEC. Он выполняет построение зависимостей между - CAPEC (Common Attack Pattern Enumeration and Classification) - шаблонами атак; - MITRE ATT&CK - методами тактик; - CWE (Common Weakness Enumeration) - классификацией уязвимостей; - CVE (Common Vulnerabilities and Exposures) - эмпирических зарегистрированных уязвимостей. Построение и визуализация данных цепочек позволяет провести более глубокий и точный анализ интересующих специалиста угроз, сопоставляя теоретическую базу моделей проведения атак с эмпирическими уязвимостями, собираемыми в течение всего существования сообщества. В конечном счете, разработка вспомогательного модуля позволяет учесть большинство потребностей специалиста и повысить эффективность анализа и дальнейших расчетов.

Поскольку разработка не является исключительно инфраструктурной и предусматривает различные сценарии взаимодействия с пользователем, в качестве

важного требования к разработке выделяется наличие интерактивного и удобного, с точки зрения пользовательского опыта, GUI – графического интерфейса пользователя. Единство оформления и лаконичность, в совокупности с оптимизированным под нужды специалиста расположением графических элементов, должны обеспечить высокий пользовательский опыт и удобство использования, повышая эффективность работы над анализом безопасности системы.

На финальных этапах разработки необходимо провести комплексное тестирование системы, что позволит выявить баги и ошибки исполнения. Для этого разработанное решение покрывается комплексом “unit” и компонентных тестов, что позволяет выполнять отладку на всех абстрактных уровнях, включая графический интерфейс пользователя, и минимизировать количество ошибок в финальной версии разрабатываемого программного обеспечения. Стоит отметить, что отсутствие непредсказуемого поведения и ошибок в работе программы также положительно сказывается на пользовательском опыте и напрямую влияет на эффективность использования.

Итоговое решение должно позволять пользователю взаимодействовать через графический интерфейс со всеми компонентами системы, включая бесперебойный доступ к базе данных, корректное исполнение алгоритмов через графический интерфейс и визуализацию полученных результатов.

3.2 Выбор технологий и инструментов

В этом разделе освещена тема выбора технологий и инструментов, необходимых для реализации работы программно-технического комплекса, на рисунке 16 рассмотрим их наглядно.

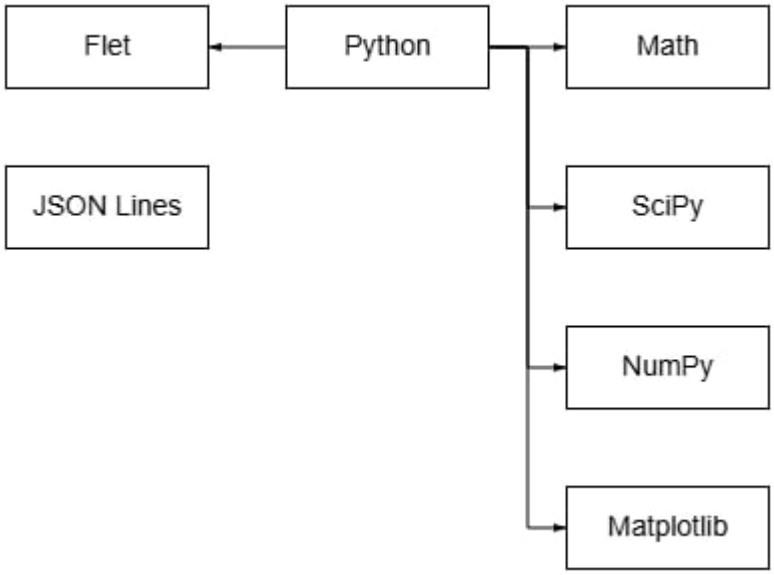


Рисунок 16 - Технологии и инструменты программно-технического комплекса

Как видно на рисунке 16 основным и единственным языком проекта был выбран Python. Данный язык программирования является своего рода стандартом в области анализа данных, машинного обучения и научных вычислений, тому доказательством служат его лидирующие позиции в рейтингах языков среди публикация в таких журналах, как IEEE, TIOBE и PYPL. Также Python обладает одним из самых активных и больших сообществ разработчиков и исследователей в мире, что означает [74-81]:

- огромное количество документации и учебных ресурсов;
- быстрое обнаружение и исправление ошибок в основных библиотеках;
- постоянное развитие языка и экосистемы.

Гибкость, выразительность и кроссплатформенность, то есть работоспособность языка программирования на Windows, Linux и macOS, Python позволяют быстро разрабатывать, совершенствовать и сопровождать модели. Нельзя не упомянуть и тот факт, что практически все необходимое для исследовательской работы на языке Python имеет открытые лицензии и таким образом позволяет свободно использовать и модифицировать исследовательский код и модели без ограничений, что является критически важным в академической среде.

Немаловажным аргументом в пользу выбора Python служит его набор открытых библиотек. Первой библиотекой использовалась стандартная библиотека math, которая предоставляет базовые математические функции и константы для работы с значениями типа int и float. Также в набор входят следующее [60-64]:

- тригонометрические функции: sin(), cos(), tan(), asin(), pi и так далее;
- гиперболические функции: sinh(), cosh() и прочие;
- экспоненциальные и логарифмические функции: exp(), log(), log10(), sqrt();
- работа с дробными числами: ceil(), floor(), fabs() и так далее;

Преимуществами данной библиотеки является ее наличие в стандартной библиотеке Python, что позволяет проводить элементарные операции над одиночными числами без дополнительных условий. Однако недостатком библиотеки math служит невозможность работать с массивами и другими математическими объектами из линейной алгебры.

В данной дипломной работе библиотека math была применена для расчетов некоторых значений в комбинаторных вычислениях, например, как было описано ранее, для расчетов больших степеней биномиального распределения использовалось логарифмирование.

Однако возможностей библиотеки math для реализации полноценной аналитической модели недостаточно. В данном случае принято применять библиотеки NumPy (Numerical Python) и SciPy (Scientific Python). Первая представляет из себя фундаментальную библиотеку для высокопроизводительных научных вычисления в Python. Ее ядром является N-мерный массив, что позволяет использовать эффективную структуру для хранения больших объемов однородных данных всех типов, например, целые, вещественные или комплексные числа. Следующие дополнительные полезные возможности библиотеки [66]:

- векторизированные математические операции - позволяют получить огромный выигрыш в скорости благодаря применению базовых математических функций сразу ко всему массиву;
- индексирование и срезы - гибкие и мощные способы доступа к данным и манипулирование массивами;

- функции линейной алгебры: np.dot() (скалярное, матричное умножение), np.linalg.inv() (обратная матрица), np.linalg.eig() (собственные значения и векторы), np.linalg.solve() (решение систем линейных алгебраических уравнений);

- генерация данных с рандомизацией по необходимым параметрам;

- агрегация по N-мерному массиву;

- трансформация массивов.

Таким образом из вышеописанных характеристик и возможностей следует, что библиотека NumPy позволяет увеличить скорость работы программы, эффективно использовать память, при этом сохраняет лаконичность синтаксиса Python и является основой целой экосистемы о библиотеках которой дальше и пойдет речь.

Библиотека SciPy является надстройкой над NumPy и входит в упомянутую ранее экосистему. На этой базе представлены алгоритмы и инструменты для продвинутых научных и инженерных вычислений путем реализации широкого инструментария для работы с массивами. Основные модули и возможности:

- распределение вероятностей;

- статистические тесты;

- оценка параметров распределений;

- описательная статистика;

- решение обыкновенных дифференциальных уравнений;

- расширение набора функций над N-мерными массивами;

- инструментарий для работы с сетевыми функциями и анализом.

Преимуществами и непосредственно поводами для использования SciPy в реализации программно-технического комплекса являются:

- моделирование частоты атак (распределение Пуассона);

- моделирование времени между атаками;

- моделирование ущерба;

- проверка гипотез;

- расчет Value-at-Risk (VaR) и Conditional VaR на основе подобранных распределений;

- подбор параметров сложных вероятностных моделей риска (максимальное правдоподобие);

- оптимизация параметров модели для минимизации ошибки прогноза;

- поиск оптимальных порогов срабатывания систем обнаружения (минимизация FPR при заданном TPR);

- калибровка моделей;

- расчет интегральных показателей риска;

- решение дифференциальных уравнений в динамических моделях распространения угроз и уязвимостей;

- эффективное представление больших графов сетевой инфраструктуры или матриц инцидентности атак;

- восстановление пропущенных значений в данных наблюдений;

- сглаживание временных рядов интенсивности атак.

В таблице 5 представлены наглядные сравнения характеристик представленных библиотек, для выбора оптимальных.

Таблица 5 - Сравнение характеристик представленных библиотек

Характеристика	Math	NumPy	SciPy
Уровень	Базовый - скалярные значения	Фундаментальный - массивы данных	Прикладной - алгоритмы
Фокус	Простые функции	Эффективные массивы, линейная алгебра, базовая статистика	Сложная математическая статистика, оптимизация, интегрирование
Типы данных	int, float	N-мерные массивы	N-мерные массивы

Таблица 5 - продолжение

Характеристика	Math	NumPy	SciPy
Производительность	Низкая Циклическая реализация	Высокая Векторная реализация	Высокая Оптимизированные алгоритмы
Использование	Минимальное, элементарное	Обязательно для данных и базовых расчетов	Обязательно для анализа
Зависимость	Стандартная библиотека	Требует установки	Требует NumPy

Следующей из экосистемы NumPy использовалась библиотека Matplotlib - фундаментальная библиотека для научной визуализации в Python. Данный инструментарий позволяет визуализировать все необходимые расчеты в виде графиков и диаграмм.

Визуализация позволяет увидеть закономерности, аномальные точки и взаимосвязи в данных и расчетах в отличие от равномерного числового выхода программы. В рамках данной дипломной работы с помощью Matplotlib переводятся в графический формат результаты всех аналитических расчетов [64-70].

Возможности библиотеки позволяют создавать графики качества, подходящего под публикации, пояснительные записки и презентационный материал. Основной инструментарий Matplotlib представлен ниже.

Базовые типы графиков, для которых используется модуль pyplot являющийся основным интерфейсом:

- линейные графики. вызываются командой plt.plot:

- позволяют отражать зависимости от времени, например, частоту атак или уровень риска, а также динамики изменений метрик и результаты итераций модели;

- столбчатые диаграммы. вызываются командами plt.bar, plt.barh.

- применяются наглядного сравнения по разным типам метрик, например по видам атак, например ddos, фишинг и ransomware, или распределение инцидентов по источникам;

- гистограммы вызываются командой plt.hist:

- данный вид визуализации позволяет отследить распределение ключевых величин по времени, что является необходимым в анализе рисков;

- круговые диаграммы, команда plt.pie:

- необходимый для отображения долей разных составляющих внутри одного исследования, например, количество атак на объекты разного типа;

- диаграммы размаха - boxplots, plt.boxplot:

- предоставляют наглядное сравнение распределений для разных сценариев, временных периодов и систем, позволяют выявить типичные значения и аномалии.

Продвинутая визуализация для глубокого анализа- диаграммы рассеяния, выявление корреляций или их отсутствия между параметрами. Например, связь между сложностью атаки и наносимым ущербом, связь времени обнаружения и масштаба инцидента.

Тепловые карты визуализация матриц корреляций между факторами риска, карты интенсивности атак по времени суток/дням недели, представление данных в разрезе двух категорий.

Графики плотности распределения - сглаженное представление распределения величины (например, ущерба) вместо ступенчатой гистограммы. Позволяет лучше оценить форму, особенно "тяжелые хвосты" (риск редких, но катастрофических событий). Очень важно для моделей риска!

Графики с несколькими осями Y - совмещение на одном графике разных, но связанных метрик. Например, частота атак (левая ось) и средний ущерб (правая ось) по типам атак.

Ключевые преимущества Matplotlib для исследовательских работ - гибкость и контроль визуализации.

Можно настроить абсолютно все - размер, шрифт, цвет, стиль, маркеры, легенды, подписи осей с помощью LaTeX для формул, сетку, пределы, описания и комментарии [64-70].

Интеграция с NumPy и SciPy -простое построение графиков из N-мерных массивов NumPy и визуализация результатов расчетов scipy.stats.

Разные форматы вывода информации - сохранение графиков в высококачественных векторных форматах, таких как PDF и SVG, для интеграции в материал без потери качества или растровых форматах PNG и JPG для презентационного материала.

Поддержка сложных композиций - возможность создавать несколько связанных графиков разных типов в одном пространстве, с целью отобразить все необходимые зависимости для наиболее информативного отображения.

Широкая совместимость и основа для других библиотек - Matplotlib как и NumPy стала базой для таких библиотек, как seaborn и графики в pandas, где представлен расширенный набор инструментов.

Еще одним инструментов в разработке программно-технического комплекса стала Plotly — высокоуровневая библиотека для создания интерактивной научной визуализации на Python, основанная на JavaScript-библиотеке D3.js [64-70]..

Научные аспекты Plotly:

- возможность зумирования, панорамирования, выделения областей на графиках;
- динамическое отображение данных при наведении курсора позволяет исследовать аномалии в распределении кибератак;
- поддержка сложных пространственных моделей 3d-визуализация сетевых топологий, траекторий распространения угроз;
- готовые шаблоны для box-plot с группировкой по типам атак, pca-проекций многомерных данных угроз, визуализации доверительных интервалов в моделях риска.

В таблице 6 будут описаны примеры использования графиков из фреймворка Flet [64-71].

Таблица 6 - Примеры применений графиков из Plotly

Тип графика	Применение в кибербезопасности
Интерактивные временные ряды	Анализ трендов атак
Тепловые карты	Визуализация матриц, корреляция факторов риска
Графы	Отображение своей между
Геокарты	Распределение источников атак по странам
Анимированные графики	Динамика изменения риска во времени

Технические преимущества библиотеки Plotly для научной визуализации [64-70]:

- Plotly обеспечивает глубокую совместимость с ключевыми инструментами экосистемы Python, а именно нативную поддержку структур данных NumPy и pandas через высокоуровневый модуль plotly.express;

- прямая визуализация результатов математического моделирования из SciPy, включая распределения вероятностей;

- интерактивные веб-форматы, например, HTML-экспорт для интеграции в веб-приложения и Jupyter Notebook;

- возможность выводить графики в векторные форматы SVG и PDF для научных публикаций и отчетов;

- совместимость с фреймворком Dash для создания аналитических панелей управления.

Архитектура библиотеки основана на принципах четкого разделения данных и их визуального представления, иерархической организации графических элементов через объекты Figure и Traces и автоматизации рутинных задач компоновки.

Ключевым отличием Plotly от статистических библиотек является двусторонняя коммуникация с данными, что превращает визуализацию в инструмент

анализа, а не просто в средство графического представления результатов. В таблице 7 рассмотрим сравнение Plotly и Matplotlib.

Таблица 7 - Сравнение Plotly и Matplotlib [75-78]

Критерий	Plotly	Matplotlib
Интерактивность	Нативная	Требует доп.библиотек
3D-визуализация	Качественный рендер	Базовые возможности
Сложные компоновки	subplots, dashboards	-
Публикационное качество	Векторный экспорт	Полный контроль
Порог вхождения	Требуется изучение API	Просто для базовых задач

JSON Lines (JSONL) — простой и современный формат хранения и передачи потоковых данных и больших объемов данных, характеризующийся тем, что каждый объект JSON записывается в отдельную строку файла без общего массива и разделительных запятых. Этот ряд подходов дает принципиальные преимущества, особенно актуальные для кибербезопасности обработки данных и масштабных информационных систем. Формат JSONL поддерживает потоковую обработку, делает чтение и запись данных быстрыми, что приводит к необходимости загрузки всего файла в память, что критично при работе с регистрами кибератак, объем которых может занимать несколько терабайт. В отличие от классического JSON, требующего загрузки и обработки всего документа, JSONL позволяет эффективно работать со строками, существенно снижая требования к оперативной памяти [81].

Кроме того, формат включает добавление новых записей, так как новые объекты JSON просто добавляются в конец файла за одну операцию, без необходимости повторного сохранения всего файла. Это повышает устойчивость к ошибкам: если файл поврежден, правильно написанные строки перед поврежденным

разделом всегда считываются, в отличие от обычного JSON, где повреждение целевого раздела делает весь документ недействительным. JSONL также обеспечивает высокую совместимость с командными строками Unix, такими как grep, sed и awk, что позволяет интегрироваться с различными конвейерами обработки данных и позволяет быстро фильтровать и преобразовывать данные [81].

Важным преимуществом является широкая поддержка форм в современных базах данных и аналитических инструментах: MongoDB, Elasticsearch и других платформах, имеющих встроенные возможности импорта и экспорта данных в формате JSONL без дополнительных преобразований.

Использование формы JSONL соответствует нескольким направлениям. Во-первых, это логирование и мониторинг, в частности в области кибербезопасности, где JSONL используется для потоковой передачи событий от IDS/IPS, межсетевых экранов и SIEM-систем. Например, запись журнала кибератак в реальном времени может включать такие поля, как source_ip, target, type и risk_score, что позволяет эффективно агрегировать и анализировать данные. Во-вторых, этот формат широко используется в экосистемах больших данных для процессов ETL, являясь эталоном в таких технологиях, как Apache Spark. JSONL используется для экспорта и импорта данных между различными хранилищами, например, при миграции данных из MongoDB в Elasticsearch, а также для создания дампов данных и резервных копий [81].

Таким образом, формат JSON Lines является универсальным, эффективным и надежным решением для организации, хранения и передачи потоковых и больших данных в современной среде, особенно в такой критической области, как кибербезопасность. Для сравнения была составлена таблица 8, в которой подробно проведено сравнение JSONL с другими форматами хранения данных.

Таблица 8 - Сравнение JSONL с другими форматами

Критерий	JSONL	JSON	CSV
Структура	Один объект на строку	Весь файл - массив	Таблица с заголовками
Большие файлы	Быстрая обработка	Медленная обработка	Средняя обработка
Гибкость схемы	Средняя	Высокая	Средняя
Добавление данных	Добавление	Перезапись	Добавление

Для наглядной демонстрации разработанной в данной работе модели оценки риска кибератак было создано интерактивное приложение с использованием фреймворка Flet — современного инструмента для построения кроссплатформенных графических интерфейсов на Python.

Flet — ориентированный на Python фреймворк, позволяющий создавать интерактивные графические интерфейсы (GUI) с единым кодом для различных платформ: веб-браузеров, настольных ОС (Windows, macOS, Linux) и мобильных устройств (через Flet App). В отличие от традиционных решений, Flet не требует знания HTML, CSS или JavaScript — все приложение реализовано полностью на Python. Интерфейс построен на основе декларативной модели виджетов, аналогичной концепции Flutter, что позволяет реализовывать интуитивно понятные и отзывчивые пользовательские интерфейсы [55-74].

Среди ключевых особенностей Flet следует выделить следующие:

- единая кодовая база для всех платформ;
- горячая перезагрузка при изменении кода (реактивная разработка);

- интеграция с экосистемой Python (NumPy, Pandas, Matplotlib, Scikit-learn и т. д.);

- придерживание принципов паттерна Observer;

- асинхронная обработка задач (async/await), что критично для операций, требующих значительных вычислительных ресурсов;

- богатая библиотека встроенных компонентов, включая таблицы, графики, диалоги, панели навигации и инструменты визуализации данных (PlotlyChart, Image, DataTable и т. д.).

Основной целью разработки интерфейса в данной работе было обеспечение интерактивной визуализации модели оценки киберрисков, а также демонстрация ее возможностей в режиме реального времени на различных устройствах. Приложение позволяет пользователю:

- выбирать сценарии атак из предопределенного набора;

- задавать параметры входных данных (частота атак, ущерб, вероятность) через элементы интерфейса;

- визуализировать результаты в виде интерактивных графиков и таблиц.

Таким образом, приложение не только делает работу более понятной для неподготовленной аудитории, но и позволяет представить модель как потенциально полезный аналитический инструмент в реальной практике оценки киберрисков.

Использование Flet дало следующие преимущества:

- интеграция с библиотеками Python, используемыми в работе (например, расчеты с помощью NumPy и визуализация результатов через Matplotlib и Plotly).

- быстрая разработка и прототипирование, что позволяет сосредоточиться на логике модели без лишней привязки.

- кроссплатформенность и универсальность развертывания — приложение может быть представлено как веб-сервис или как локальный исполняемый файл.

- интерактивность интерфейса, недостижимая при использовании статических презентаций или отчетов.

- расширяемость — добавление моделирования атак.

3.3 Компоненты пользовательского интерфейса

Реализация спроектированной архитектуры основана на общепринятых парадигмах и методологиях, успешно применяющихся в лучших практиках при разработке программного обеспечения [60]. В первую очередь, компоненты системы распределены на слои, обеспечивающие функционал, аналогичный инструменту “namespace” – пространство имен – из языков, основанных на парадигмах классического объектно-ориентированного программирования: такой подход позволяет не только разделить ответственность компонентов, но и придерживаться единства в наименовании объектов общего назначения, что улучшает читаемость и повышает прозрачность логики использования в различных частях приложения [55-80].

Проект включает в себя слои, ответственность которых разделена следующим образом:

- core: слой, включающий в себя все компоненты, ответственные за бизнес-логику приложения. В нем описаны и реализованы математические модели для расчетов рисков, используемые для построения зависимостей.

- plotter: слой, содержащий компоненты, ответственные за построение и экспорт изображений в пользовательский интерфейс, гарантируя отсутствие логики расчетов.

- gui: слой, хранящий в себе абстракции графических компонентов и глобального состояния приложения; поставляет изолированные друг от друга компоненты, необходимые для построения отдельных частей интерфейса пользователя и его сборки воедино.

- main: модуль точки входа в приложение; главный компонент, инициализирующий интерфейс и отвечающий за логику обработки событий, генерируемых взаимодействием пользователя и программы.

Принцип разделенной ответственности позволяет разработчику с легкостью тестировать отдельные компоненты приложения и добавлять новые, не нарушая архитектурную целостность. Разделенная ответственность также предлагает изоляцию одних компонентов от других, что обеспечивает легкость управления с

помощью настраиваемых и, что немаловажно, контролируемых разработчиком зависимостей: примером таковых могут являться компоненты, взаимодействующие с интерфейсом операционной системы, глобальные состояния и т. д.

Особенности реализации абстракции каждой из частей позволяют применять различные архитектурные паттерны, удовлетворяющие условиям функционирования определенного компонента. В реализации используются различные паттерны проектирования, перенятые как части философии используемых фреймворков, так и внедренные самостоятельно для обеспечения решений, необходимых для взаимодействия [70-82].

Используемый графический фреймворк “flet” использует сразу несколько паттернов, позволяющих сохранять гибкость и удобство для разработчика. Паттерн “Composite” позволяет собирать объекты интерфейса пользователя из нескольких атомарных “контролов” – простейших компонентов, что обеспечивает простоту построения нагруженных интерфейсов, подчиняющихся свойствам главного окна. Примером такого взаимодействия служит конструктор компонентов, описывающих композицию и распределение “контролов” в главном окне: объект “Column”, используемый для размещения построчно, принимает в своем конструкторе список атомарных объектов, после чего монтируется на страницу. Аналогом “Composite”, не для описания общей структуры, но для описания самих объектов, является парадигма “Declarative-UI”, позволяющая гибко настраивать “контролы” не списком последовательных команд, а передачей конкретных свойств в соответствующем конструкторе. Кроме того, “flet” активно использует паттерн “Observer”, что обеспечивает корректное функционирование обработчиков взаимодействия с пользователем – клавиши, слайдеры и текстовые поля, используемые для создания форм, содержат в своих конструкторах поле-ссылку на обработчики, которые вызываются при совершении “события”, в данном случае являющегося результатом взаимодействия пользователя системой. Таким образом изменение состояния гарантируется тогда и только тогда, когда пользователь взаимодействует с компонентами, гарантируя отсутствие случайных вызовов [70-82].

Помимо паттернов, являющихся частью философии самого фреймворка “flet”, были применены отдельные практики, улучшающие опыт разработчика и позволяющие избегать неопределенного поведения и ошибок. Как упомянуто выше, для обеспечения изоляции компонентов и разделения ответственности между ними используется компонентный подход проектирования, что подразумевает отсутствие всяческого функционала для обращения к полям классов, содержащих элементы пользовательского интерфейса и используемых ими данных. Для решения этой проблемы был использован паттерн “Dependency Injection”, позволяющий собирать необходимые для корректной работы общего пайплайна данных в одно глобальное состояние и передавать его по ссылке в конструкторы компонентов графического интерфейса: такой подход позволяет безопасно обмениваться данными между изолированными частями и с легкостью расширять имеющийся функционал [70-82].

Для описания глобального состояния используется отдельный класс, хранящий в себе все необходимые публичные поля, а также набор сеттеров для множественного присваивания при сбросе без удаления контекста в рамках одной рабочей сессии. Чтобы гарантировать единственность такого объекта используется паттерн “Singleton” – такой подход защищает программиста от создания дополнительного дублирующего объекта, и при повторной инициализации возвращает ссылку на уже существующий.

Схема абстрактного взаимодействия компонентов в одном из модулей представлена на рисунке 17.

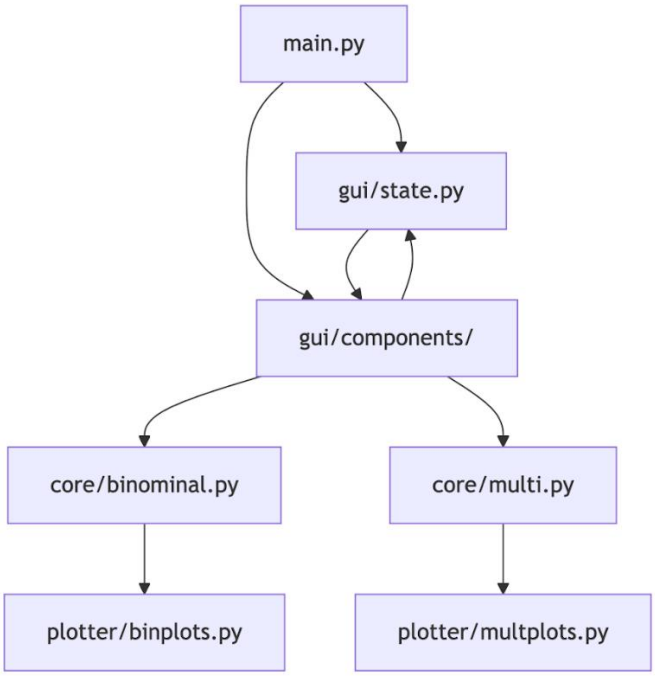


Рисунок 17 – Взаимодействие между модулями системы

Перейдем от общей абстракции взаимодействия компонентов к частным реализациям и разберем логику и структуры, используемые для решения прикладных задач, необходимых для расчета рисков кибератак и их визуализации.

3.3.1 Ввод и обработка идентификаторов уязвимостей

Разработанная система является многомодульной, с условием сохранения принципа разделения ответственности. Перед началом работы, набор дополнительного прикладного ПО актуализирует данные об уязвимостях, обновляя данные в корневом файле “jsonl”. Модуль, позволяющий рассчитывать метрики и по выбранной тактике для одиночных атак и рассчитывать по ней метрики реализован с учетом принципов, описанных выше. Общий алгоритм расчета и оценки одиночной атаки укладывается в один небольшой модуль, использующий одну форму для ввода и поле для форматирования данных, в котором отображаются загруженные из базы данных данные. Поскольку для хранения данных используется файловая не реляционная БД в формате “jsonl”, распаковка и загрузка ее в ОЗУ, с последующей

инициализацией объекта python с ссылкой на эту область происходит с помощью встроенной библиотеки “json”, поставляющей декодер.

Интерфейс модуля представлен атомарными “контролами” (“TextField”, “ElevatedButton”, “Text”, “Progress Bar”, “DataTable”, “ListView” и др.), оформленными через “контролы” раскладок в “Column” и “Row”. После ввода пользователем данных, если таковые присутствуют для дальнейшей работы программы, система ожидает вызова соответствующей клавиши, обработчик которой запускает общий алгоритм обработки. Расчет и прогнозирование для набора одиночных атак, соответствующих выбираемой в вводимом поле технике производится после завершения поисков соответствий по обновленным базам данных. Программа выводит таблицу с топ-20 результатов поиска по параметру с рассчитанными метриками. Помимо прочего, для найденного множества предусмотрен сложный “контрол” для интерактивного отображения рисков в виде столбчатой диаграммы, реализованной с помощью plotly: интерактивность позволяет получить информацию без имплементации колбеков, поскольку данный функционал берет на себя сама библиотека. Веб-основы “flet” и “plotly” позволяют смонтировать интерактивный результат на главную страницу посредством атомарных “контролов”, таких как, например, “Container”.

В отображаемой информации, помимо подробных топ-20, находится полный список уязвимостей, размеченный цветовым выделением для всех рассчитанных метрик. Схематичный сценарий взаимодействия пользователя с модулем одиночной атаки изображена на рисунке 17:



Рисунок 17 – Блок-схема запроса об уязвимостях по тактике

3.3.2 Построение сценариев атак

Для построения сценариев атак, соответствуя схеме MITRE, предусмотрен отдельный программный модуль, позволяющий в интерактивном режиме построить сценарий, содержащий несколько векторов атак, используя различные категории техник, формируя их в набор множеств, содержащих необходимые уязвимости. Интерактивное взаимодействие с пользователем обеспечивается “контролами” “Stack” с соответствующими клавишами, для формирования дополнительного выбора параметров для множества указанных на предыдущем этапе техник. С помощью “контрола” “Dialog” реализована возможность выбирать значения техник и категорий для соответствующего вектора в пространстве сценария. Подобная практика позволяет не создавать дополнительные формы для ввода в теле корневого элемента – главной страницы –, а рендерить элементы поверх нее, используя свойства “z-index”, позволяющие в парадигме “flet” рендерить элементы поверх

смонтированных на главной странице “контролов”, экономя пространство внутри окна и обеспечивая доступ к форме выбора независимо от доступного места среди видимых “контролов” нулевой высоты. Дополнительные проверки на наличие уже включенных в сценарий категорий и тактик обеспечивают уникальность сценария и защиту от дублирования значений для расчетов и дальнейшего импорта в визуализатор, обеспечивая дополнительное удобство при использовании.

Этап выбора техник в сценарии закономерно переходит в режим ввода соответствующих уязвимостей, используемых для определения направленности атаки указанной техники. Ввод данных об уязвимостях обеспечивает форма, рендеринг которой обеспечивает нажатие соответствующей клавиши в основном меню: форма содержит в себе поля из уже знакомых “контролов” “TextField” с соответствующими проверками на формат введенной уязвимости и автоматически генерирует достаточный набор таких “контролов” для каждой указанной пользователем в конструкторе тактики. Введенные значения вводятся в соответствующий объект и вызывается отдельным обработчиком клавиши экспорта, позволяющей сохранить сценарий в отдельный файл “jsonl”, содержащий набор векторов атаки со всеми введенными данными.

Экспорт файла осуществляется отдельным методом, гарантирующим корректность и единство формата файла-сценария, структура которого едина и для остальных модулей системы. Пользовательский путь внутри реализованного модуля схематично изображен на рисунке 18:



Рисунок 18 – Блох-схема создания сценария атаки

3.4 Модуль визуализации

3.4.1 Вывод графов

Модуль для визуализации графов реализует интерактивный инструмент для визуализации связей между CAPEC, MITRE ATT&CK техниками, CWE и CVE. Он спроектирован с целью демонстрации и дальнейшего исследования связей сценариев атак с техниками, типами уязвимостей и конкретными уязвимостями, в частности. Зона ответственности отображаемой информации также использует принцип

разделенной ответственности: помимо атомарных “контролов”, в данной части приложения используются класс-конструктор, которому делегируется корректное отображение направленного графа связей между выбранными единицами. Взаимодействие с пользователями осуществляется с помощью “контролов” “TextField”, “ElevatedButton” и “Slider”, что позволяет управлять параметрами отображаемого графа. Помимо прочего, в пользовательском интерфейсе предусмотрено интерактивное взаимодействие в виде отображаемых уязвимостей, соответствующих ребру связей – по клику на линию ребра графа появляется дополнительное всплывающее окно, содержащее в себе текстовый список всех связанных уязвимостей для улучшения пользовательского опыта и возможности быстрого экспорта полученной информации в остальные компоненты системы. Схематичное отображение пользовательского сценария изображено на рисунке 19:

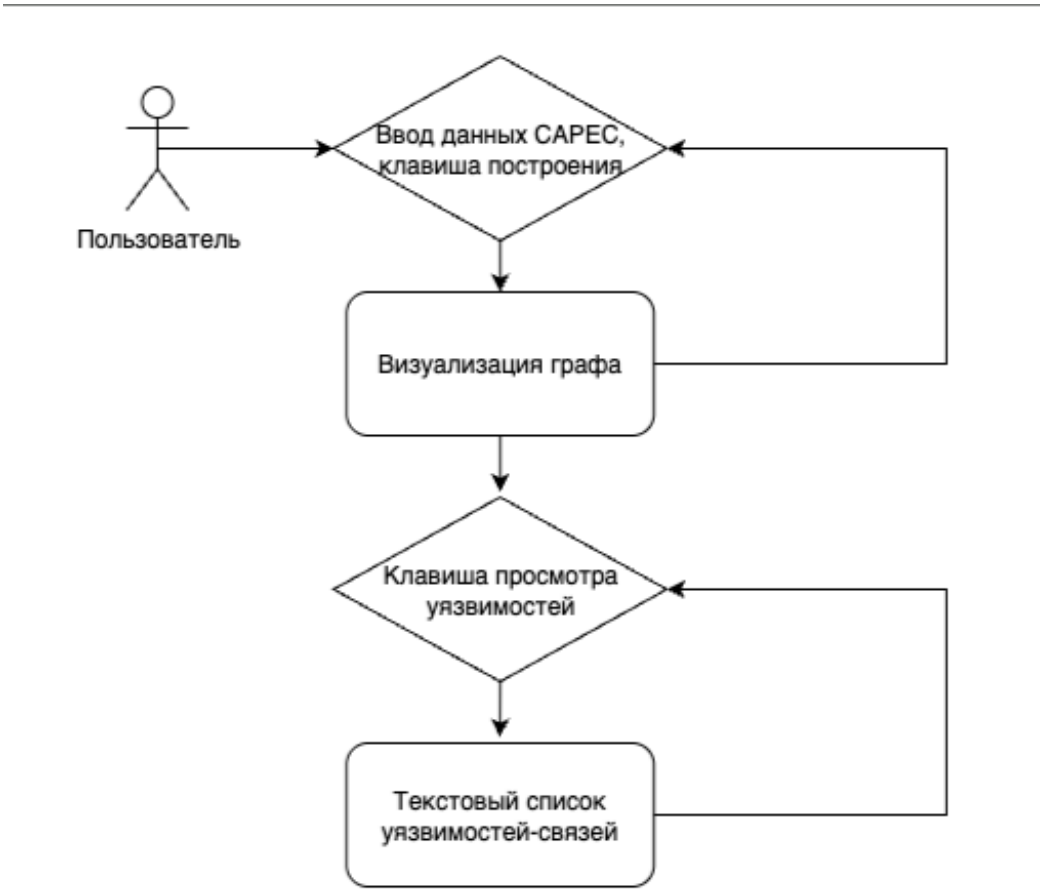


Рисунок 19 – Схема взаимодействия для отрисовки графов связей

3.4.2 Вывод графиков и взаимодействие с ними

Модуль визуализации и расчета рисков атак выполнен с учетом всех практик и парадигм, описанных в реализации. Данный модуль является самым наиболее нагруженным из всех, так как сочетает в себе функционал расчетов и визуализации нескольких видов атак: множественных и многовекторных, к которым соответственно применяются разные подходы к вычислениям и визуализации полученных результатов.

Точкой входа в приложение является логический слой “main”, инициализирующий все объекты пользовательского интерфейса, а также поставляющий объекты управления глобальным состоянием приложения и взаимодействия с файловым браузером через интерфейс операционной системы. Также в этом слое находятся ограниченное количество импортов, что поддерживает архитектурный подход разделения ответственности – при его использовании гарантируется отсутствие циклических импортов, где функционал различных компонентов и модулей заимствует методы чужих модулей.

Конструктор пользовательского интерфейса в модуле “main” включает в себя всего два основных кастомных и композитных “контрола”: интерактивный поиск уязвимостей и вкладки, определяющие функционал для каждого из вариантов расчета, поставляющие соответственно различные множества форм и клавиш для управления данными и расчетами.

Все математические модели и расчеты объявлены в пространстве “core”: соответствующие каждому виду атак математические модели и вычисления изолированы от других частей приложения и вызываются в содержимых соответствующих вкладок. Математическая реализация состоит из имплементации алгоритмической части, описанной главой выше: реализация на матпакетах обеспечивает скорость работы и корректную обработку граничных случаев: использование логарифмических выражений для оптимизации факториальных вычислений, комбинаторных и вероятностных, поставляется с помощью библиотек numpy и scipy, продемонстрировавших высокую производительность на количестве

итераций, соответствующих граничному для корректного диапазона расчета рисков, релевантного для проведения дальнейшего анализа [70-82].

Интерактивный поиск по уязвимостям вынесен в отдельный класс пространства “gui”: он принимает в конструкторе указатель на объект глобального состояния, предоставляющий доступ к полю, хранящему все записи об уязвимостях из локальной базы данных, представленной все тем же “jsonl”, а также к полю, хранящему в себе текущее выбранное множество уязвимостей, использующихся для дальнейших расчетов. Уникальным “контролом” в этом компоненте можно считать динамически обновляемый ListView, предоставляющий возможность выбрать первые пять подходящих для поиска вариантов, указанных пользователем в TextField. Набор выбранных уязвимостей не только не ограничен, но и интерактивно позволяет собой управлять посредством “контрола” “Chip”, список с которыми отображается под “контролом” поля для поиска: каждый из них имеет клавишу удаления, соответствующе связанную с полем выбранных для анализа уязвимостей.

Композитный “контрол” с вкладками имеет вложенную структуру, управление которой реализовано отдельным классом “TabView”, включающим в себя инициализацию каждой вкладки с функционалом, в конструкторы которых, как по цепочке вызовов, передается ссылка на объект глобального состояния. Поскольку класс унаследован от “контрола” “Tabs”, все его поля и методы доступны для вызовов и перегрузки, что обеспечивает гибкое управление и надежное разделение имен с отдельными классами, что позволяет делать декларативные поля отдельными именами и выборочно перегружать их в зависимости от поведения. Это позволяет адаптировать интерфейс к различным сценариям использования и потенциальному расширению, если при дальнейшей работе возникнет необходимость добавлять новые вкладки и функционал [70-82].

Содержимое каждой вкладки, соответствующее решениям для каждого из случаев видов атак, вынесено в отдельные классы, унаследованные от “контрола” “Column”: это, аналогично классу-сборщику табов, позволяет адаптировать интерфейс, но еще и назначать расположение “контролов” внутри содержимого без дополнительной “обертки” – атомарные “контролы” внутри класса такого вида по

умолчанию занимают равномерное пространство по вертикали. Для вкладки с множественными атаками предусмотрено поле для ввода количества атак, берущееся в расчет, а также еще один композитный “контрол” “Row”, содержащий в себе мок графика, представленный статическим изображением из ассетов, и панель управления, состоящую из “контрола” “Column”, включающего в себя клавиши и слайдеры, изменяющие параметры расчета управления атаками. Колбеки, предусмотренные для элементов пользовательского интерфейса, по изменению пользовательского состояния, соответственно интерактивно отображают информацию (слайдеры) и запускают алгоритм расчета и очистку текущего состояния. Слайдеры, в свою очередь, поставляют функционал управления рисками и обеспечивают возможность динамически просматривать применяемые изменения, что повышает удобство пользователя и позволяет наглядно зафиксировать необходимые для ограничения рисков параметры. Содержимое вкладки для многовекторных атак устроено совершенно аналогично, однако, вместо слайдеров, содержит в себе дополнительное поле для ввода коэффициента снижения наносимого ущерба и возможность выбирать несколько вариантов использования программы, поставляя как функционал чтения сценария из файла, генерируемого дополнительным сепарированным модулем, так и возможность выбирать уязвимости, указанные с помощью интерактивного поиска. Реализовано такое решение с помощью “контрола” “RadioGroup”, ограничивающего пользователя единственностью выбора из представленного множества вариантов. Выбор файла сценария реализован с помощью “контрола” “FilePicker” – решение из “flet”, позволяющее, как указано выше, методами операционной системы выбрать файл с ограничением размера выборки и формата. Каждое действие на обработчиках сопровождается вызовом “контролов” “Snack”, уведомляющих пользователя об успехе/неуспехе совершаемых операций с информацией, рендер которых происходит по принципу “z-index”, описанного выше [70-82].

Каждый обработчик событий использует соответствующие алгоритмы построения визуализаций, логика которых изолирована от остальных частей программы. Поскольку каждый вид атак имеет разные подходы к визуализации,

функции для построения графиков вынесены в пространство “plotter”, методы которого принимают на вход рассчитанные в “core” значения и возвращает графики в едином формате, но с соответствующим содержанием: кривые распределений для множественных атак и столбчатую диаграмму для многовекторных. Схематическое изображение взаимодействия пользователя с модулем интерфейса представлено на рисунке 20.

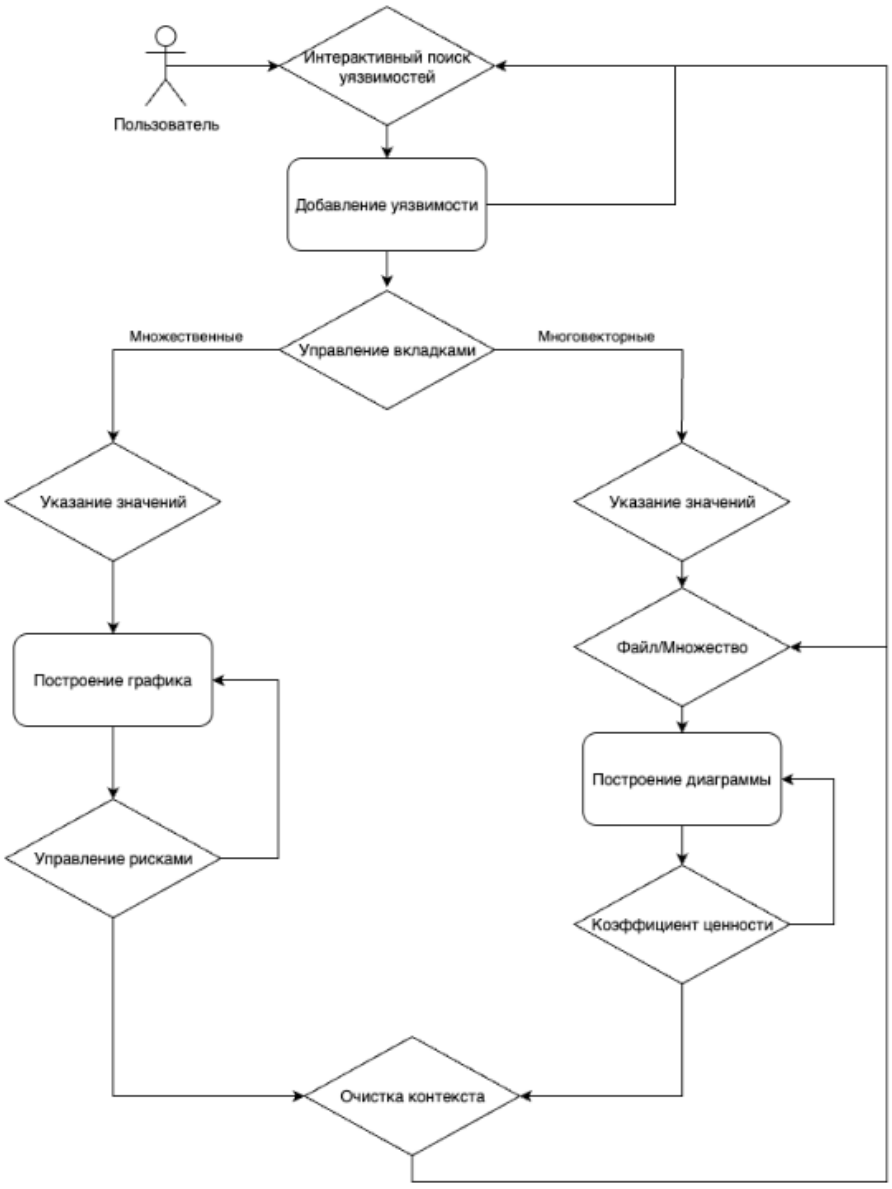


Рисунок 20 – схема сценария использования модуля визуализации

3.4.3 Вывод списков информации

Вспомогательный модуль просмотра данных об одной выбранной CVE аналогичен по своей структуре и подходу к модулю описания одиночной атаки. В нем схожим образом используются атомарные “контролы”, за исключением того, что форма вывода обеспечивается работой “контрола” “Text”, который последовательно собирает поля для вывода в программе посредством поставляемых в стандартной библиотеке Python форматных строк. Общий вид пользовательского взаимодействия представлен на рисунке 21.



Рисунок 21 – блок-схема обработки одной атаки

3.6 Ключевые направления для будущей доработки

В условиях стремительно растущей сложности и многообразия киберугроз особое значение приобретает создание высокоэффективных инструментов анализа и

оценки рисков кибератак. Приложение, разработанное на языке программирования Python с использованием фреймворка Flet, представляет собой инновационную платформу с уникальным потенциалом для дальнейшего развития и адаптации к быстро меняющейся среде информационной безопасности.

Во-первых, выбор Python в качестве базового языка программирования обусловлен его фундаментальными преимуществами с точки зрения универсальности, модульности и богатой экосистемы специализированных библиотек. Сюда входят мощные инструменты для научных вычислений (NumPy, SciPy), анализа данных (Pandas), машинного обучения (scikit-learn, TensorFlow, PyTorch), а также инструменты для визуализации и создания интерактивных интерфейсов. Такая гибкость позволяет исследователям и разработчикам быстро внедрять новые алгоритмы и методы анализа киберугроз, что имеет решающее значение для адекватного реагирования на развивающиеся атаки.

С архитектурной точки зрения фреймворк Flet предоставляет эффективный механизм кроссплатформенной разработки, предоставляя единую кодовую базу для настольных и веб-приложений [75]. Это существенно упрощает миграцию программного продукта в веб-пространство, расширяя доступ к аналитическим инструментам через браузер, без необходимости установки локального ПО. Такая веб-ориентированность облегчает интеграцию приложения в корпоративные и облачные инфраструктуры, облегчая коллективное использование и масштабирование решения. Важной перспективой является возможность облачного развертывания с использованием контейнеризации и микросервисной архитектуры, что повысит отказоустойчивость, масштабируемость и безопасность платформы.

С технической точки зрения дальнейшее развитие аналитических возможностей приложения предполагает внедрение более сложных и реалистичных сценариев моделирования кибератак. Комбинированные и комплексные сценарии, отражающие многовекторные и многоэтапные атаки (см. рис 22), требуют разработки сложных моделей взаимодействия различных угроз и их корреляции. Использование формальных методов описания сценариев атак и построения графов угроз позволит

проводить более точную и подробную оценку рисков, что критически важно для принятия решений в области управления информационной безопасностью.



Рисунок 22 - Структура видов кибератак

Ключевым направлением развития является интеграция методов машинного обучения и искусственного интеллекта, направленная на автоматизацию и повышение точности анализа. Использование алгоритмов глубокого обучения, рекуррентных и графовых нейронных сетей позволит моделировать сложные временные и структурные зависимости от данных кибератак, выявлять скрытые закономерности и прогнозировать потенциальные угрозы с высокой степенью достоверности. Особое внимание следует уделить адаптивным и самообучающимся системам, способным учитывать динамические изменения поведения злоумышленников и быстро меняющуюся тактику атак.

Кроме того, использование методов Explainable AI повысит доверие конечных пользователей и специалистов по безопасности к полученным аналитическим результатам, обеспечивая прозрачность и интерпретируемость моделей. Это имеет решающее значение для практического внедрения аналитических систем в корпоративной среде, где решения должны подкрепляться обоснованными

выводами и подтверждаться аудитами. Наконец, перспективы интеграции приложения с внешними системами и источниками данных, включая платформы SIEM, IDS/IPS и Threat Intelligence, открывают возможность создания единой экосистемы информационной безопасности с автоматизированным обменом знаниями и быстрым реагированием на инциденты. Таким образом, развитие платформы будет способствовать не только повышению эффективности анализа рисков, но и формированию проактивной модели защиты на основе комплексного мониторинга и глубокого понимания современного ландшафта киберугроз.

В совокупности комплексный подход, основанный на гибкости и расширяемости Python и Flet, интеграция современных методов искусственного интеллекта и глубокое моделирование сложных сценариев атак открывают перспективы создания инновационного аналитического инструмента, способного адаптироваться к вызовам будущего и существенно повысить уровень информационной безопасности на различных уровнях организации.

ЗАКЛЮЧЕНИЕ

В ходе работы **цель исследования** – разработка эффективных алгоритмов расчета рисков кибератак и программно-технического комплекса, реализующего их – достигнута.

Решенные задачи:

1) Обзор и существующих подходов к риск-моделированию, были кратко рассмотрены существующие предложения, направленные на риск-моделирование кибератак, для выбора направления развития работы;

2) Разработка алгоритмов:

- построения связанной базы данных;
- автоматического обновления метрик для уязвимостей;
- расчета рисков для единичных, множественных и многовекторных атак;
- управления риском для множественных и многовекторных атак.

3) Программная реализация, был реализован программно-технический комплекс на основе разработанных алгоритмов с модулями визуализации расчетов, а также со вспомогательными модулями изучения связей и уязвимостей.

Практическая значимость:

- комплекс может применяться в аналитической работе специалистов по информационной безопасности для оценки рисков кибератак;
- решение можно интегрировать со сканерами уязвимостей для увеличения функционала и возможностей комплекса;
- разработанные алгоритмы упрощают анализ сложных атак, что сокращает время принятия решения на меры по реагированию.

В качестве **перспектив развития** для дальнейшего улучшения программного комплекса предлагается:

1) Расширение функциональности через разработку дополнительных модулей или интеграций с уже имеющимися решениями как из разряда обнаружения, изучения и расчета инцидентов кибератак, так и из разряда противодействия им;

2) Разработка автоматизированной комбинации атак и их автоматизированного расчета, достижение этого результата потребует разработки инновационных алгоритмов мульти поточности и огромных вычислительных ресурсов, так как количество вариаций будет настолько велико, что с ним будет справиться по силу только суперкомпьютеру.

Итоговые выводы:

Разработанные алгоритмы обеспечивают:

- создание связанной базы данных для детального изучения уязвимостей;
- оценку рисков кибератак с учетом данных об их уязвимостях;
- управление риском для множественных и многовекторных сценариев атак.

Рекомендации:

- провести комплексное тестирование в государственных компаниях, отвечающих за информационную безопасность;
- адаптировать комплекс под использование в организациях;
- продолжить разработку в сторону полной автоматизации комплекс, где вершиной будет являться комплекс, самостоятельно реагирующий на любые изменения в контексте уязвимостей информационной безопасности.

СПИСОК ЛИТЕРАТУРЫ

1. Эпидемии в телекоммуникационных сетях / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2020. – 282 с. (Серия «Теория сетевых войн»; вып. 1). – Текст: непосредственный.

2. Атакуемые взвешенные сети / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2020. – 248 с. (Серия «Теория сетевых войн»; вып. 2). – Текст: непосредственный.

3. Социальные сети и деструктивный контент / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2020. – 274 с. (Серия «Теория сетевых войн»; вып. 3). – Текст: непосредственный.

4. Социальные сети и риск-мониторинг / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2020. – 266 с. (Серия «Теория сетевых войн»; вып. 4). – Текст: непосредственный.

5. Социальные сети и психологическая безопасность / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2021. – 232 с. (Серия «Теория сетевых войн»; вып. 5). – Текст: непосредственный.

6. Сетео-информационная эпидемиология / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2021. – 216 с. (Серия «Теория сетевых войн»; вып. 6). – Текст: непосредственный.

7. Картография защищаемого киберпространства / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2022. – 372 с. (Серия «Теория сетевых войн»; вып. 7). – Текст: непосредственный.

8. Организационно-правовая защита сетей / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2023. – 228 с. (Серия «Теория сетевых войн»; вып. 8). – Текст: непосредственный.

9. Автоматизированный атлас кибератак / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2025. – 188 с. (Серия «Теория сетевых войн»; вып. 9). – Текст: непосредственный.

10. Модели кибератак // TAdviser. - URL: https://www.tadviser.ru/index.php/Сатья:Модели_кибератак

11. Васильев В. И., Вульфин А. М., Кириллова А. Д., Кучкарова Н. В. Методика оценки актуальных угроз и уязвимостей на основе когнитивного моделирования и Text Mining // SCCS Intelgr. 2021. - URL: https://sccs.intelgr.com/archive/2021-03/04-Vasilyev.pdf - Текст: электронный.

12. Модели кибератак. Систематизируем подходы к представлению // SecurityLab, 15.07.2020 - URL: https://www.securitylab.ru/analytics/510135.php - Текст: электронный.

13. Hutchins D. R., Cloppert M. J., Amin R. M. Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains, 2011 - URL: https://www.lockheedmartin.com/content/dam/lockheed-martin/rms/documents/cyber/LM-White-Paper-Intel-Driven-Defense.pdf - Текст: электронный.

14. MITRE ATT&CK Framework. - URL: https://attack.mitre.org - Текст: электронный.

15. NIST SP 800-115. Technical Guide to Information Security Testing and Assessment - URL: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf - Текст: электронный.

16. Certified Ethical Hacker (CEH) Course Outline // EC-Council. - URL: https://www.eccouncil.org/programs/certified-ethical-hacker-ceh/ - Текст: электронный.

17. CAPEC – Common Attack Pattern Enumeration and Classification. - URL: https://capec.mitre.org - Текст: электронный.

18. CVE and CVSS Guide // NVD – National Vulnerability Database. - URL: https://nvd.nist.gov - Текст: электронный.

19. Common Weakness Enumeration (CWE) // MITRE. - URL: https://cwe.mitre.org - Текст: электронный.

20. Применение методов агрегации в информационных системах // Scilead.ru - URL: https://scilead.ru/article/4868-primenenie-metodov-agregatsii-v-informatsionnyh-sistemah - Текст: электронный.

21. Агрегация данных: ключевые методы для эффективного анализа // Wiki Analytics. - URL: https://sky.pro/wiki/analytics/agregatsiya-dannyh-klyuchevye-metody-dlya-effektivnogo-analiza/ - Текст: электронный.

22. Всё об управлении уязвимостями в 2025: Vulnerability Management // Infra-Tech. - URL: https://blog.infra-tech.ru/upravlenie-uyazvimostyami/ - Текст: электронный.

23. Обзор баз данных угроз // Security Vision. URL: https://www.securityvision.ru/blog/obzor-baz-dannykh-ugroz/ - Текст: электронный.

24. Передупреждён — значит вооружён: подборка отчётных ресурсов с инф. об угрозах // Habr, VAS Experts. - URL: https://habr.com/ru/companies/vasexperts/articles/790192/ - Текст: электронный.

25. “Turning the database inside-out with Apache Samza” (2015) – URL: https://martin.kleppmann.com/ - Текст: электронный.

26. Dean J., Ghemawat S. MapReduce: simplified data processing on large clusters // Commun. ACM. 2008. - URL: https://www.researchgate.net/publication/325574460_MapReduce_Simplified_Data_Processing_on_Large_Cluster - Текст: электронный.

27. Сафиуллина А. Н. Оценка параметров биномиального распределения по методу моментов и её асимптотические свойства // Ученые записки Казанского университета. Серия физико-математические науки. — 2016. — Т. 158, № 2. — С. 221–230. - URL: https://cyberleninka.ru/article/n/otsenka-parametrov-binomialnogo-raspredeleniya-po-metodu-momentov-i-ee-asimptoticheskie-svoystva - Текст: электронный.

28. Воробьёв Н. Н. *Многомерные дискретные распределения* // Учебное пособие. — Красноярск: СФУ, 2007. — 42 с. - URL: https://elib.sfu-kras.ru/bitstream/handle/2311/619/vorob%27ev.pdf - Текст: электронный.

29. Гмурман В. Е. Теория вероятностей и математическая статистика. — 9-е изд., перераб. и доп. — М.: Высшая школа, 2003. — 479 с. (Раздел: Квантильные характеристики распределений) - Текст: непосредственный.

30. Гаврилова А. С. Метод Монте-Карло в моделировании // Форум молодых ученых. — 2019. — № 3(29). — С. 45–48. - URL: https://cyberleninka.ru/article/n/metod-monte-karlo-v-modelirovanii - Текст: электронный.

31. Веденин Д. В. О современном состоянии киберпреступности в Российской Федерации // Вестник Уральского юридического института МВД России. 2024. № 2. С. 140–145. - URL: https://cyberleninka.ru/article/n/o-sovremennom-sostoyanii-kiberprestupnosti-v-rossiyskoy-federatsii/viewer - Текст: электронный.

32. CVSS v2 - URL: https://bdu.fstec.ru/calc - Текст: электронный.

33. CVSS v3.1 - URL: https://bdu.fstec.ru/calc31 - Текст: электронный.

34. CVSS v4 - URL: https://bdu.fstec.ru/calc4 - Текст: электронный.

35. EPSS калькулятор - URL: https://theowni.github.io/EPSS-Calculator/ - Текст: электронный.

36. OWASP калькулятор - URL: https://www.owasp-risk-rating.com/ - Текст: электронный.

37. Столбова Н. А. Характеристика личности преступника, совершающего незаконный оборот оружия с использованием информационно-телекоммуникационных сетей, в том числе сети Интернет // Российский следователь. 2023. № 10. С. 51–53. - URL: https://lawinfo.ru/en/articles/4482/xarakteristika-licnosti-prestupnika-soversayushhego-nezakonnyi-oborot-oruziya-s-ispolzovaniem-informacionno-telekommunikacionnyx-setei-v-tom-cisle-seti-internet - Текст: электронный.

38. Овсюков Д. А. Корыстные преступления против собственности с использованием информационнотелекоммуникационных сетей: вопросы квалификации: монография / под науч. ред. С. М. Кочои. М.: Проспект, 2023. 184 с. – Текст: непосредственный.

39. Бутырская А. В., Шувалова А. М., Бубнова О. Ю. Актуальные вопросы противодействия преступлениям, совершаемым с использованием информационно-телекоммуникационных сетей // Российский следователь. 2022. № 12. С. 58–63. - URL: https://elibrary.ru/item.asp?edn=ylrllx - Текст: электронный.

40. Глазкова Л. В. Преступления экстремистской направленности, совершаемые с использованием сферы телекоммуникаций и компьютерной информации // Актуальные проблемы российского права. 2021. № 12. С. 167–176 - URL: https://cyberleninka.ru/article/n/prestupleniya-ekstremistskoy-napravlennosti-sovers

haemye-s-ispolzovaniem-sfery-telekommunikatsiy-i-kompyuternoy-informatsii - Текст: электронный.

41. Кибератаки на российские компании в 2024 году - URL: https://rt-solar.ru/analytics/reports/5320/ - Текст: электронный.

42. Методы моделирования атак - URL: https://ptresearch.media/articles/metody-modelirovaniya-atak - Текст: электронный.

43. VulnCheck - URL: https://vulncheck.com/?gad_source=1 - Текст: электронный.

44. CISA KEV (Known Exploited Vulnerabilities) - URL: https://www.cisa.gov/resources-tools/resources/kev-catalog - Текст: электронный.

45. Андерсон Р. Инженерия безопасности: руководство по построению надёжных распределённых систем / Росс Андерсон. — 2-е изд. — Москва : Горячая линия — Телеком, 2012. — 944 с. — ISBN 978-5-9912-0976-2. — Текст: непосредственный.

46. Bitsight Reveals More than 60 Percent of Known Exploited Vulnerabilities Remain Unmitigated Past Deadlines in First-of-its-Kind Analysis of CISA’s KEV Catalog - URL: https://www.bitsight.com/press-releases/bitsight-reveals-more-60-percent-known-exploited-vulnerabilities-remain-unmitigated?utm_source=Securitylab.ru - Текст: электронный.

47. Шелупанов А. А. Форензика: теория и практика расследования киберпреступлений / А. А. Шелупанов, А. Р. Смолина. — Москва : Горячая линия — Телеком, 2018. — 104 с. — ISBN 978-5-9912-0769-0. — Текст: непосредственный.

48. Сабанов А. Г., Шелупанов А. А. Идентификация и аутентификация в цифровом мире / А. Г. Сабанов, А. А. Шелупанов. — Москва : Горячая линия — Телеком, 2022. — 356 с. — ISBN 978-5-9912-0976-2. — Текст: непосредственный.

49. Мещеряков Р. В., Шелупанов А. А. Комплексное обеспечение информационной безопасности автоматизированных систем / Р. В. Мещеряков, А. А. Шелупанов. — Томск : В-Спектр, 2007. — 350 с. — ISBN 978-5-91191-182-5. — Текст: непосредственный.

50. Зайцев А. П., Шелупанов А. А., Мещеряков Р. В. Технические средства и методы защиты информации: учебник для вузов / под ред. А. П. Зайцева, А. А.

Шелупанова. — 7-е изд., испр. и доп. — Москва : Горячая линия — Телеком, 2012. — 425 с. — ISBN 978-5-9912-0084-4. — Текст: непосредственный.

51. Корченко О., Василий Е., Гнатюк С. Современные квантовые технологии информационной безопасности / Oleksandr Korchenko, Yevhen Vasiliu, Sergiy Gnatyuk. — Харьков : Изд-во ХНУ, 2010. — 152 с. — ISBN 978-966-123456-7. — Текст: непосредственный.

52. Ageeva V., Novokhrestov A., Kholodova M. Угрозы информационной системе в физической среде и киберпространстве / Valeria Ageeva, Aleksey Novokhrestov, Maria Kholodova. — Москва : ИБС, 2020. — 204 с. — ISBN 978-5-0000-0001-2. — Текст: непосредственный.

53. Clarke R. Cyberwar: file under fiction / Richard Clarke. — New York : Ecco, 2010. — 384 p. — ISBN 978-0-06-196919-7. — Текст: непосредственный.

54. Perlroth N. This Is How They Tell Me the World Ends: The Cyberweapons Arms Race / Nicole Perlroth. — New York : Bloomsbury, 2021. — 400 p. — ISBN 978-1-632-86628-2. — Текст: непосредственный.

55. Buchanan B. The Hacker and the State: Cyber Attacks and the New Normal of Geopolitics / Ben Buchanan. — Cambridge : Harvard Univ. Press, 2020. — 300 p. — ISBN 978-0-674-85499-5. — Текст: непосредственный.

56. Python Software Foundation. The Python Tutorial / Python.org. — URL: https://docs.python.org/3/tutorial/ — Текст: электронный.

57. Python Software Foundation. Python Language Reference / Python.org. — URL: https://docs.python.org/3/reference/ — Текст: электронный.

58. Python Software Foundation. Python Standard Library / Python.org. — URL: https://docs.python.org/3/library/ — Текст: электронный.

59. Python Software Foundation. Python HOWTOs / Python.org. — URL: https://docs.python.org/3/howto/ — Текст: электронный.

60. Python Software Foundation. Python FAQ / Python.org. — URL: https://docs.python.org/3/faq/ — Текст: электронный.

61. Python Software Foundation. Installing Python Modules / Python.org. — URL: https://packaging.python.org/tutorials/installing-packages/ — Текст: электронный.

62. Python Software Foundation. Distributing Python Modules / Python.org. — URL: https://packaging.python.org/tutorials/packaging-projects/ — Текст: электронный.

63. Python Software Foundation. PEP 8 — Style Guide for Python Code / Python.org. — URL: https://www.python.org/dev/peps/pep-0008/ — Текст: электронный.

64. Python Software Foundation. PEP 257 — Docstring Conventions / Python.org. — URL: https://www.python.org/dev/peps/pep-0257/ — Текст: электронный.

65. Python Software Foundation. PEP 484 — Type Hints / Python.org. — URL: https://www.python.org/dev/peps/pep-0484/ — Текст: электронный.

66. Mark Lutz. Dive Into Python 3 / readthedocs.io. — URL: https://diveintopython3.problemsolving.io/ — Текст: электронный.

67. Allen B. Downey. Think Python 2nd Edition / greenteapress.com. — URL: https://greenteapress.com/wp/think-python-2e/ — Текст: электронный.

68. Charles R. Severance. Python for Everybody / py4e.com. — URL: https://www.py4e.com/book — Текст: электронный.

69. Zed A. Shaw. Learn Python the Hard Way / learnpythonthehardway.org. — URL: https://learnpythonthehardway.org/python3/ — Текст: электронный.

70. Al Sweigart. Automate the Boring Stuff with Python / automatetheboringstuff.com. — URL: https://automatetheboringstuff.com/ — Текст: электронный.

71. Eric Matthes. Python Crash Course—2nd Edition / no starch.com. — URL: https://nostarch.com/pythoncrashcourse2e — Текст: электронный.

72. Luciano Ramalho. Fluent Python Companion Site / fluentpython.com. — URL: https://www.oreilly.com/library/view/fluent-python/9781491946237/ — Текст: электронный.

73. Real Python. Tutorials / realpython.com. — URL: https://realpython.com/ — Текст: электронный.

74. Awesome Python. Curated list of Python frameworks, libraries, software / GitHub. — URL: https://github.com/vinta/awesome-python — Текст: электронный.

75. PyPI — the Python Package Index / pypi.org. — URL: https://pypi.org/ — Текст: электронный.

76. Python Software Foundation. Документация Python 3.10 на русском // docs.python.org. — URL: https://docs.python.org/ru/3.10/ — Текст: электронный.

77. «Метанит». Руководство по Python 3 // metanit.com. — URL: https://metanit.com/python/tutorial/ — Текст: электронный.

78. LearnPython.ru. Учебник Python для начинающих // learnpython.ru. — URL: https://learnpython.ru/introduction — Текст: электронный.

79. Python-Scripts.com. Python для начинающих: подробный вводный курс // python-scripts.com. — URL: https://python-scripts.com/python-introduction — Текст: электронный.

80. Habr. JSON для начинающих: что это и как использовать // habr.com. 2020. — URL: https://habr.com/ru/post/495842/ — Текст: электронный.

81. JSON.org. JSON — описание формата на русском языке // json.org. — URL: https://www.json.org/json-ru.html — Текст: электронный.

82. Habr. JSON Lines (JSONL): удобный формат для больших объёмов данных // habr.com. 2021. — URL: https://habr.com/ru/post/531580/ — Текст: электронный.

83. MDN Web Docs. Объект JSON в JavaScript (рус.) // developer.mozilla.org. — URL: https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Global_Objects/JSON — Текст: электронный.

84. Pandas Documentation. Чтение и запись JSON/JSONL в pandas (рус.) // pandas.pydata.org. — URL: https://pandas.pydata.org/docs/user_guide/io.html#json — Текст: электронный.

85. FastAPI. Приём JSONL-запросов в FastAPI (рус.) // fastapi.tiangolo.com. — URL: https://fastapi.tiangolo.com/ru/tutorial/request-files/ — Текст: электронный.

ПРИЛОЖЕНИЕ

1. Структура данных

База данных была реализована с помощью файловой базы данных jsonl. Хранение осуществляется в двух видах, а именно в связанной базе данных, где хранятся связи CAPEC→MITRE ATT&CK→CWE→CVE, на рисунке 1.1 отображено хранение связей для CAPEC 439, 442, 448, 457, 464, 465, 469, 471.


```
{"CAPEC_ID": "439", "Techniques": [{"Technique_ID": "T1195", "CWEs": [{"CWE_ID": "CWE-1269", "CVEs": ["CVE-2024-23591", "CVE-2023-5457"]}]}]}
{"CAPEC_ID": "442", "Techniques": [{"Technique_ID": "T1195.001", "CWEs": [{"CWE_ID": "CWE-506", "CVEs": ["CVE-2020-26292", "CVE-2023-2003", "CVE-2017-16207",
{"CAPEC_ID": "448", "Techniques": [{"Technique_ID": "T1027.009", "CWEs": [{"CWE_ID": "CWE-506", "CVEs": ["CVE-2020-26292", "CVE-2023-2003", "CVE-2017-16207",
{"CAPEC_ID": "457", "Techniques": [{"Technique_ID": "T1091", "CWEs": [{"CWE_ID": "CWE-1299", "CVEs": ["CVE-2025-26409", "CVE-2025-1073", "CVE-2024-47944"]}]},
{"CAPEC_ID": "464", "Techniques": [{"Technique_ID": "T1606.001", "CWEs": [{"CWE_ID": "CWE-359", "CVEs": ["CVE-2022-0482", "CVE-2023-36052", "CVE-2025-25042",
{"CAPEC_ID": "465", "Techniques": [{"Technique_ID": "T1090.001", "CWEs": [{"CWE_ID": "CWE-441", "CVEs": ["CVE-2020-25353", "CVE-2017-11149", "CVE-2022-0767",
{"CAPEC_ID": "469", "Techniques": [{"Technique_ID": "T1499.002", "CWEs": [{"CWE_ID": "CWE-770", "CVEs": ["CVE-2019-0005", "CVE-2024-35969", "CVE-2024-6600", "
{"CAPEC_ID": "471", "Techniques": [{"Technique_ID": "T1574.001", "CWEs": [{"CWE_ID": "CWE-427", "CVEs": ["CVE-2020-24755", "CVE-2022-40746", "CVE-2024-23909",
```


Рисунок 1.1 – Пример хранения связей для CAPEC 439, 442, 448, 457, 464, 465, 469, 471 в базе данных

Во втором файле каждая уязвимость хранится виде строки с набором параметров, но на рисунке 1.2 из-за слишком большой длины строки для повышения читаемости, каждая метрика уязвимости была перенесена на новую строку. На ранее упомянутом рисунке представлен пример хранения метрик для уязвимости с идентификатором CVE-2024-8586.


```
140046  {"CVE_ID": "CVE-2024-8586",
140047  "CVSS2_base_score": 6.4,
140048  "CVSS2_exploitability": 10.0,
140049  "CVSS2_impact": 4.9,
140050  "CVSS3_base_score": 6.1,
140051  "CVSS3_exploitability": 2.8,
140052  "CVSS3_impact": 2.7,
140053  "confidentialityImpact": 0.3,
140054  "integrityImpact": 0.3,
140055  "availabilityImpact": 0.0,
140056  "EPSS": 0.00046,
140057  "EPSS_percentile": 0.17714,
140058  "CVSS4_base_score": 3.8,
140059  "CVSS4_exploitability": 0.8,
140060  "CVSS4_impact": 3.8,
140061  "OWASP_LF": 4.6,
140062  "OWASP_IF": 2.5,
140063  "OWASP_risk_score": 3.6,
140064  "OWASP_risk_level": "Low"}
```


Рисунок 1.2 – Пример хранения метрик для уязвимости CVE-2024-8586 в базе данных

2. Лингвистическое обеспечение

Язык разработки программного комплекса – Python 3.13.

Вид используемой базы данных – Jsonl.

Фреймворк, используемый для реализации визуальной составляющей приложения – Flet.

Среда, в которой производилась разработка: Visual Studio Code.

3. Пример применения программного комплекса риск-моделирования кибератак

3.1 Работа функционала по обновлению и вспомогательных инструментов

Первым что встречает пользователя при запуске программного обеспечения это интерфейс, отвечающий за состояние базы данных. На рисунке 3.1.1 представлена визуальная интерпретация исполняемого элемента.

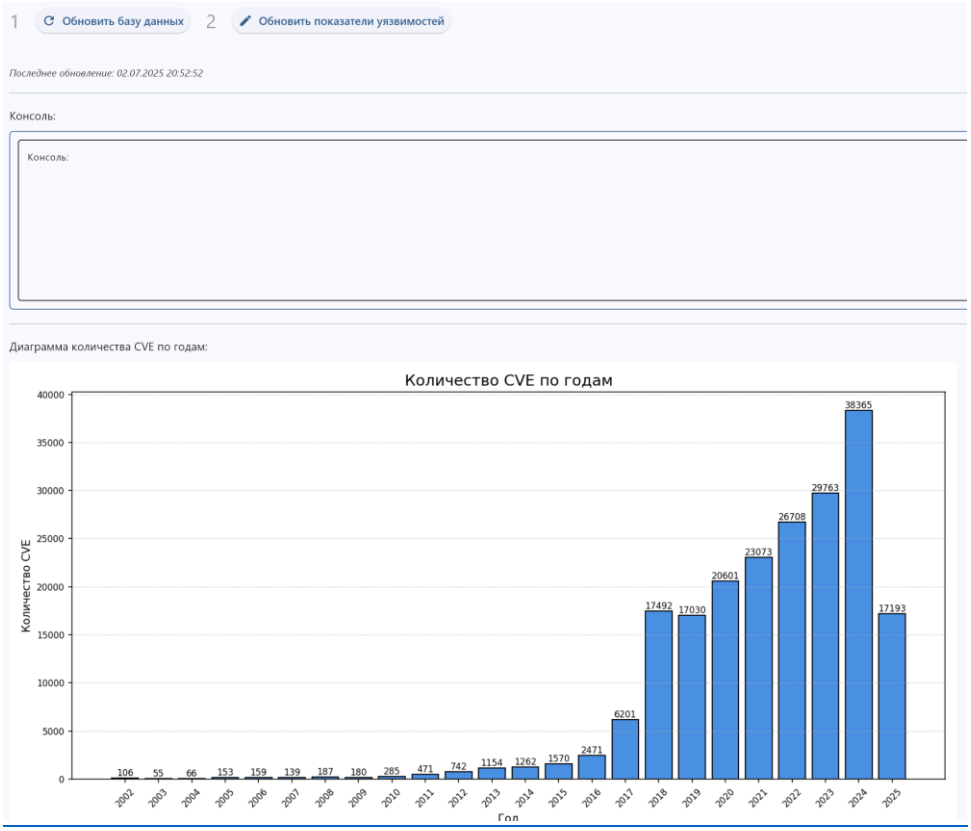


Рисунок 3.1.1 – Визуализация обновления и состояния базы данных

В левом верхнем углу находится кнопка обновления базы данных, после ее нажатия происходит обновление базы данных, а именно перестроение связанной модели CAPEC→MITRE ATT&CK→CWE→CVE, то, как она выглядит было описано ранее на рисунке 1.1.

Правее вышеописанной кнопки находится кнопка для обновления метрик каждой уязвимости, пример их хранения был отображен на рисунке 1.2.

В работе упоминалось о вспомогательных инструментах, один из таких визуализатор отображения связей CAPEC→MITRE ATT&CK→CWE→CVE через граф, так как пользователю не сильно будет понятна текстовая связь в базе данных с рисунка 1.1, поэтому и был создан этот инструмент, на рисунке 3.1.2 представлена его реализация на примере CAPEC-150.

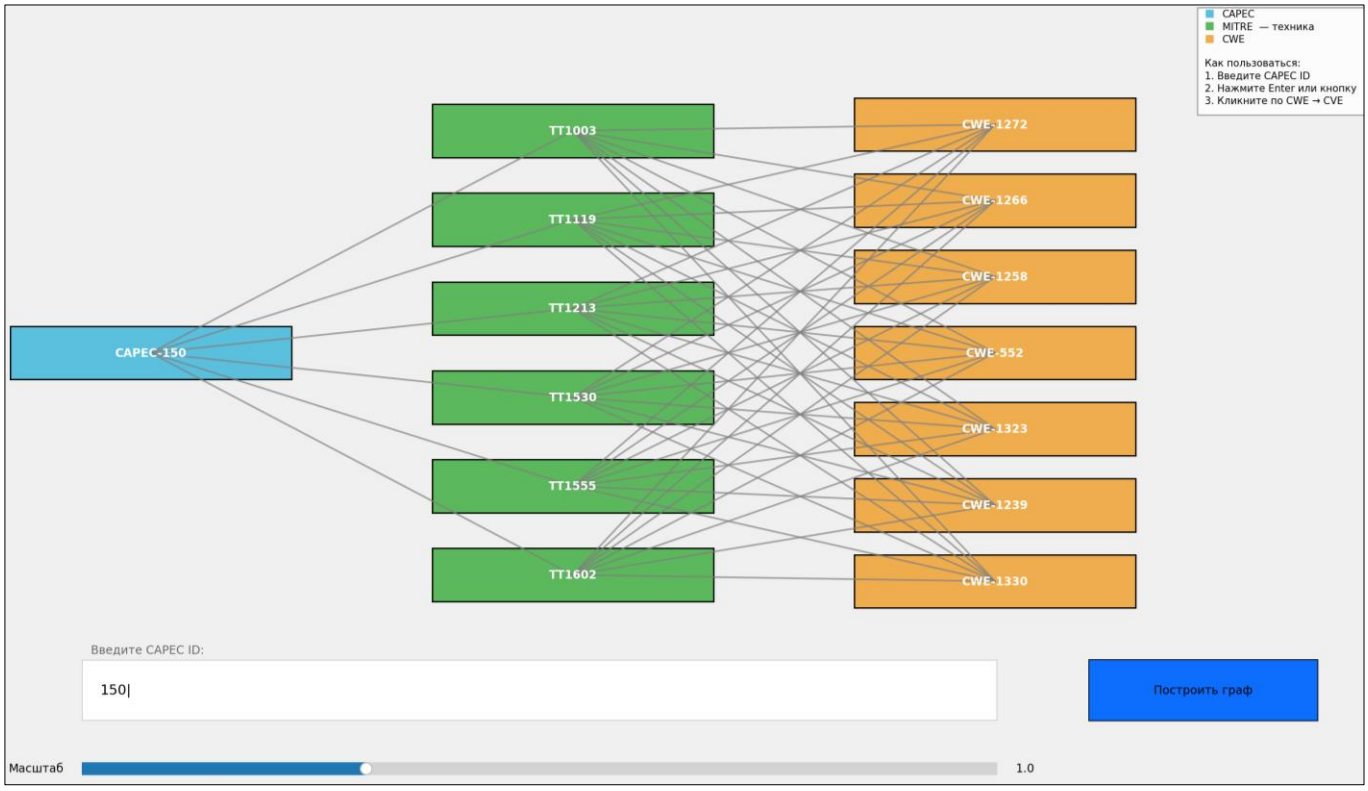


Рисунок 3.1.2 – Пример работы инструмента по отображению связей на примере CAPEC-150

Однако на вышеуказанном рисунке не хватает конечного звена CVE. Их отображение на графе было убрано специально, для повышения читаемости, ранее

описывалась эта проблема. Убрав совсем этот модуль, было бы некорректно считать работу законченной, поэтому было принято решение, сделать иконки CWE интерактивными, по нажатию на них выводится список связанных с ними CVE, это представлено на рисунке 3.1.3, по нажатию выводится весь список связанных с CWE идентификаторов уязвимостей CVE.

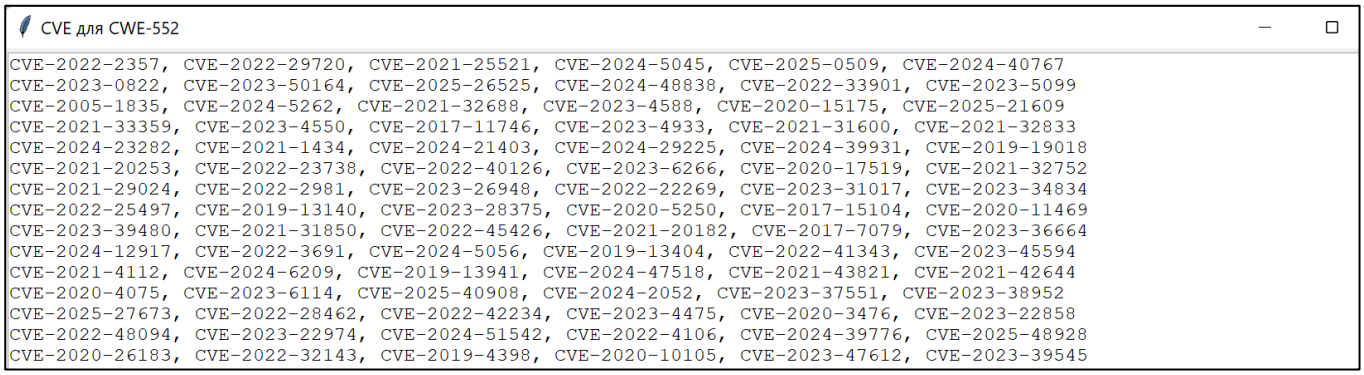


Рисунок 3.1.3 - Пример работы инструмента по отображению связей на примере CAPEC-150 после нажатия на CWE-552

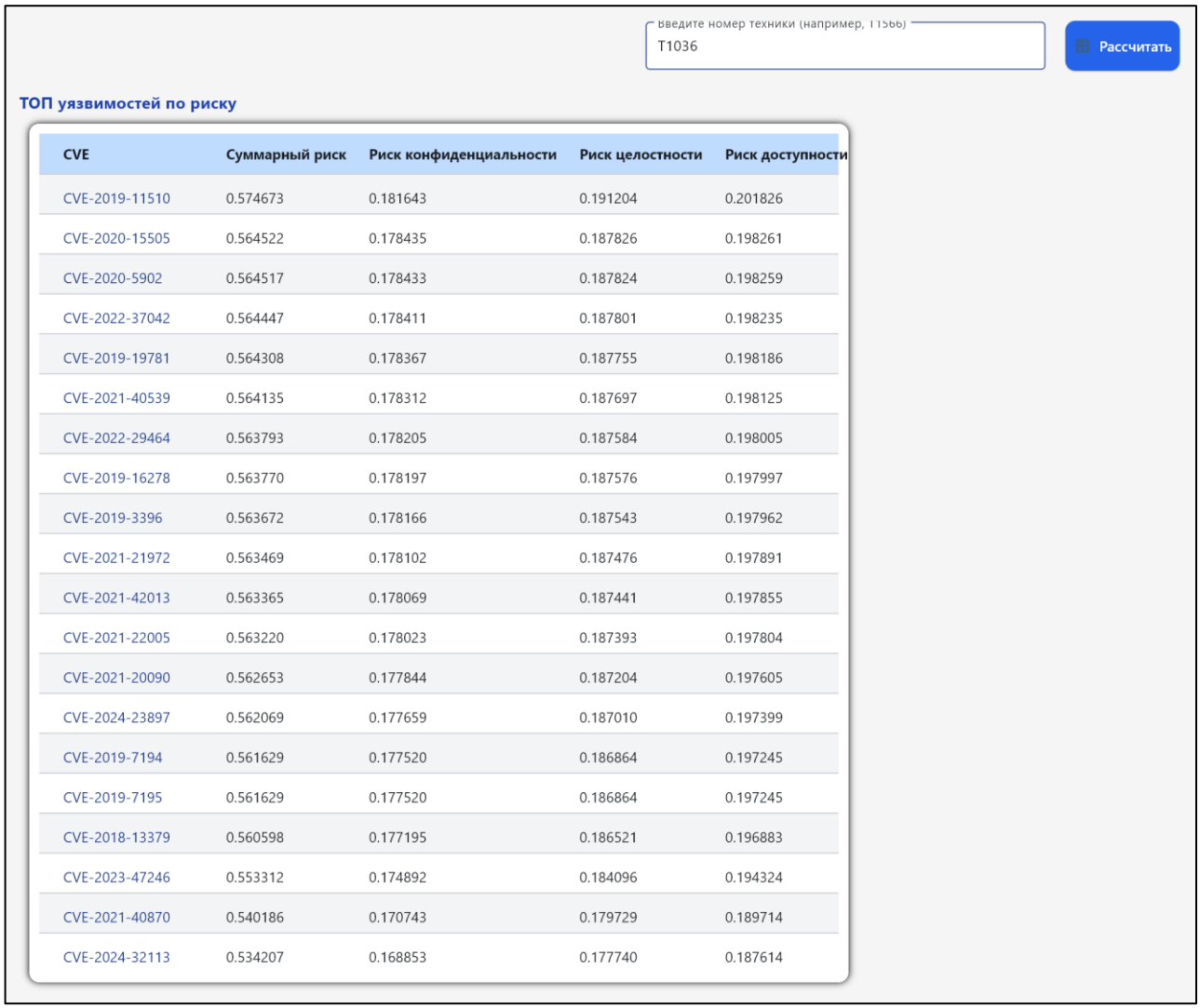
Покажем пример работы инструмента, который отображает информацию об уязвимости с рисунка 1.2, работа на примере с CVE-2022-29720 отображена на рисунке 3.1.4.



Рисунок 3.1.4 – Отображение информации об уязвимости CVE-2022-29720

3.2. Пример работы по расчёту риска для кибератак

Приведем пример для расчета рисков кибератак, начать стоит с единичных, на рисунках 3.1-3.3 показан пример расчета для уязвимостей, связанных с техникой T1036. Для каждой уязвимости рассчитываются показатели с учетом ее единичной реализации при эксплуатации в технике.



CVE	Суммарный риск	Риск конфиденциальности	Риск целостности	Риск доступности
CVE-2019-11510	0.574673	0.181643	0.191204	0.201826
CVE-2020-15505	0.564522	0.178435	0.187826	0.198261
CVE-2020-5902	0.564517	0.178433	0.187824	0.198259
CVE-2022-37042	0.564447	0.178411	0.187801	0.198235
CVE-2019-19781	0.564308	0.178367	0.187755	0.198186
CVE-2021-40539	0.564135	0.178312	0.187697	0.198125
CVE-2022-29464	0.563793	0.178205	0.187584	0.198005
CVE-2019-16278	0.563770	0.178197	0.187576	0.197997
CVE-2019-3396	0.563672	0.178166	0.187543	0.197962
CVE-2021-21972	0.563469	0.178102	0.187476	0.197891
CVE-2021-42013	0.563365	0.178069	0.187441	0.197855
CVE-2021-22005	0.563220	0.178023	0.187393	0.197804
CVE-2021-20090	0.562653	0.177844	0.187204	0.197605
CVE-2024-23897	0.562069	0.177659	0.187010	0.197399
CVE-2019-7194	0.561629	0.177520	0.186864	0.197245
CVE-2019-7195	0.561629	0.177520	0.186864	0.197245
CVE-2018-13379	0.560598	0.177195	0.186521	0.196883
CVE-2023-47246	0.553312	0.174892	0.184096	0.194324
CVE-2021-40870	0.540186	0.170743	0.179729	0.189714
CVE-2024-32113	0.534207	0.168853	0.177740	0.187614

Рисунок 3.1 – Вывод показателей для 20 уязвимостей, с наибольшими показателями, имеющие возможность эксплуатации в T1036

Далее рассмотрим визуализацию риска на рисунке 3.2 для единичных атак, она представлена в виде карты, где в порядке возрастания, слева на право выстроены уязвимости в виде столбцов, высота которых равна величине суммарного риска. При

наведении на столбец пользователю поступит информация о нем, а именно уязвимость, которую он отображает, ее суммарный риск, а также риск по компонентам (конфиденциальности, целостности, доступности). При наведении на столбец на примере вывелись расчеты по уязвимости CVE-2019-0841.

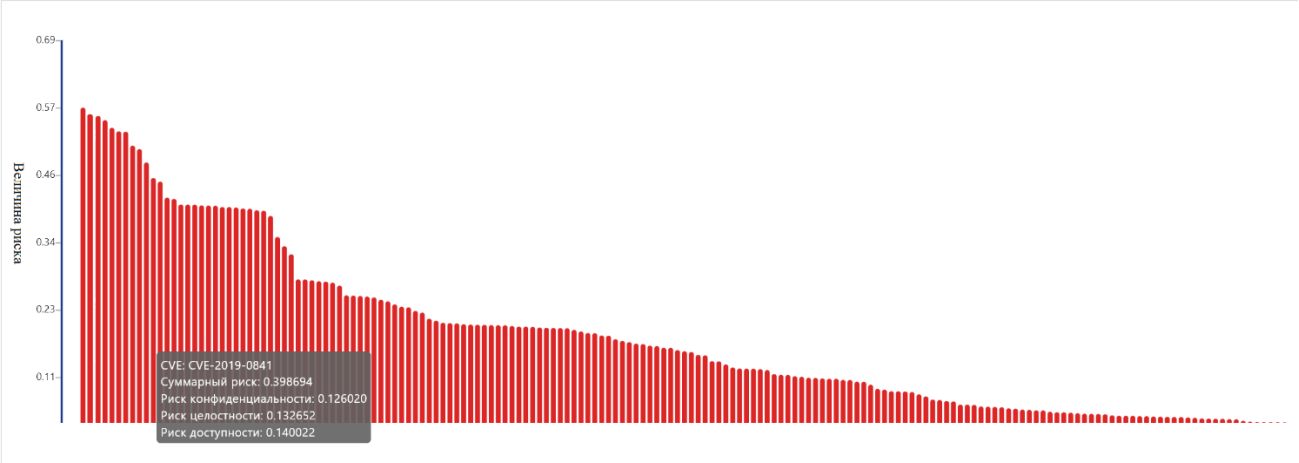


Рисунок 3.2 – Визуализация рисков для единичных атак (вывод расчетов по CVE-2019-0841)

Решено было оставить и альтернативный вариант просмотра расчетов, таковой представлен на рисунке 3.3, его показания идентичны визуализации с рисунка 3.2, но дают возможность взаимодействовать с расчетами более просто.



Рисунок 3.3 – общий список расчета для всего набора уязвимостей, связанных с T1036

Следующим рассмотрим модуль расчета риска для множественных атак с применением управления риском, на рисунке 3.4 представлен расчет атаки на систему с использованием уязвимостей CVE-2003-1605, CVE-2003-5001, CVE-2005-10001 при количестве атак равному 10000.

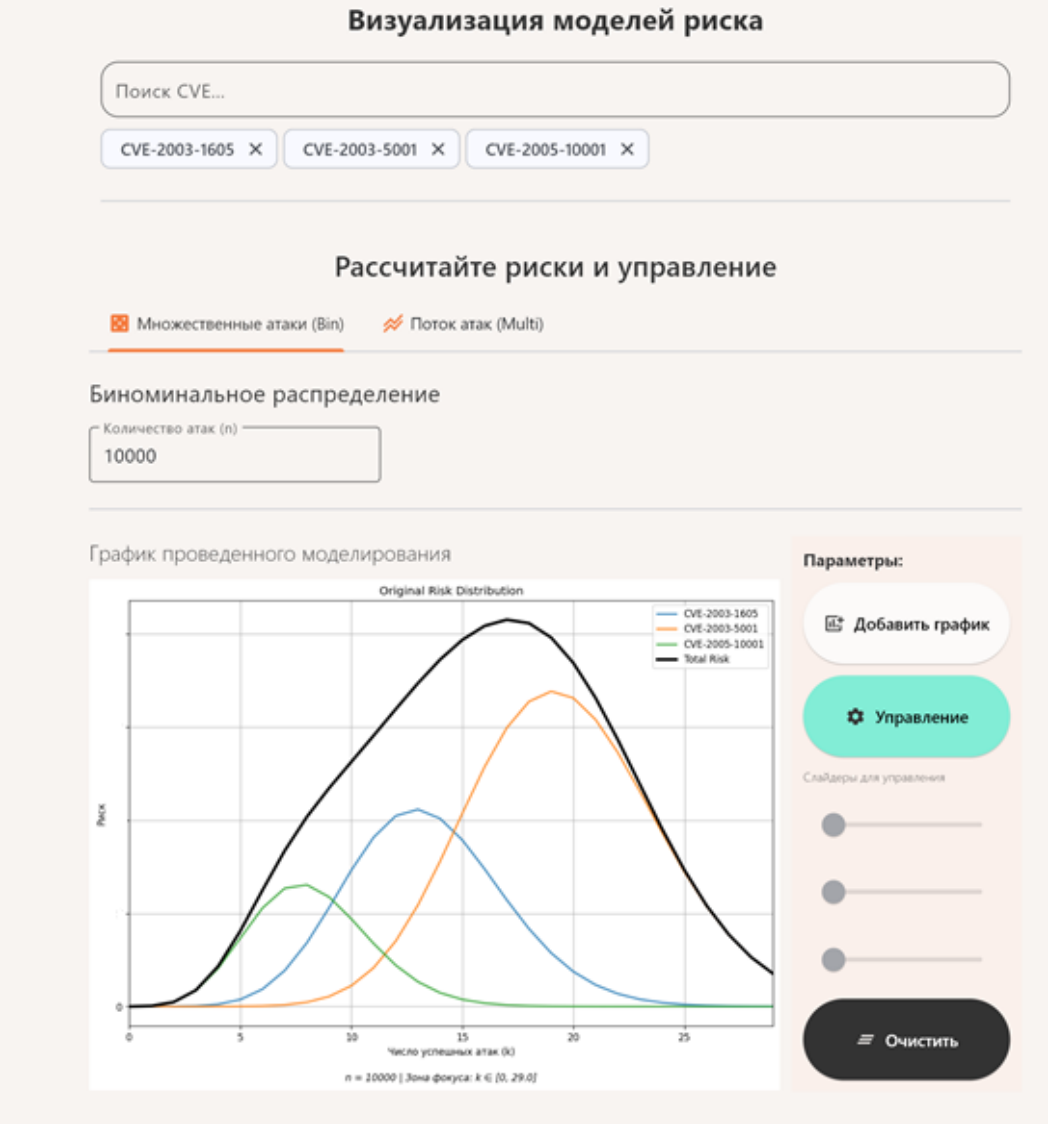


Рисунок 3.4 – Расчет показателей риска множественной атаки на систему с импользованием уязвимостей CVE-2003-1605, CVE-2003-5001, CVE-2005-10001

На вышеописанном рисунке показаны графики риска для уязвимостей, где левая с самым низким пиком CVE-2005-10001, центральная CVE-2003-1605, правая CVE-2003-5001, а над ними возвышается суммарный риск атаки на систему.

На рисунке 3.5 показан процесс управления риском для системы подверженной множественной атаке через эксплуатацию уязвимостей CVE-2003-1605, CVE-2003-5001, CVE-2005-10001 с числом атак n=10000.



Рисунок 3.5 - Процесс управления риском для системы подверженной множественной атаке через эксплуатацию уязвимостей CVE-2003-1605, CVE-2003-5001, CVE-2005-10001 с числом атак n=10000

Параметры для квантильной перенормировки вероятности:

1) Ползунок “Оценка степени угрозы” (целевой квантиль), значение на рисунке 3.4 равно 0,4;

2) Ползунок “Фокус защиты” (центр распределения), значение на рисунке 3.4 равно 0,77;

3) Ползунок “Разнородность угроз” (ширина распределения), значение на рисунке 3.4 равно 0,1;

График управления демонстрирует один из вариантов подбора параметров перенормировки, при котором суммарное функция суммарного риска лежит в заданном интервале при оптимальном управлении. Зависимость показывает отсутствие явных пиков риска при выборе разнородных уязвимостей, что удовлетворяет задаче управления риском.

Следующим рассмотрим мультиномиальное распределение, описывающее поток атак, для него нам необходимо сначала задать сценарии атак, для этого используется модуль позволяющий это сделать, для простоты использования он схож с матрицей Mitre Attack.

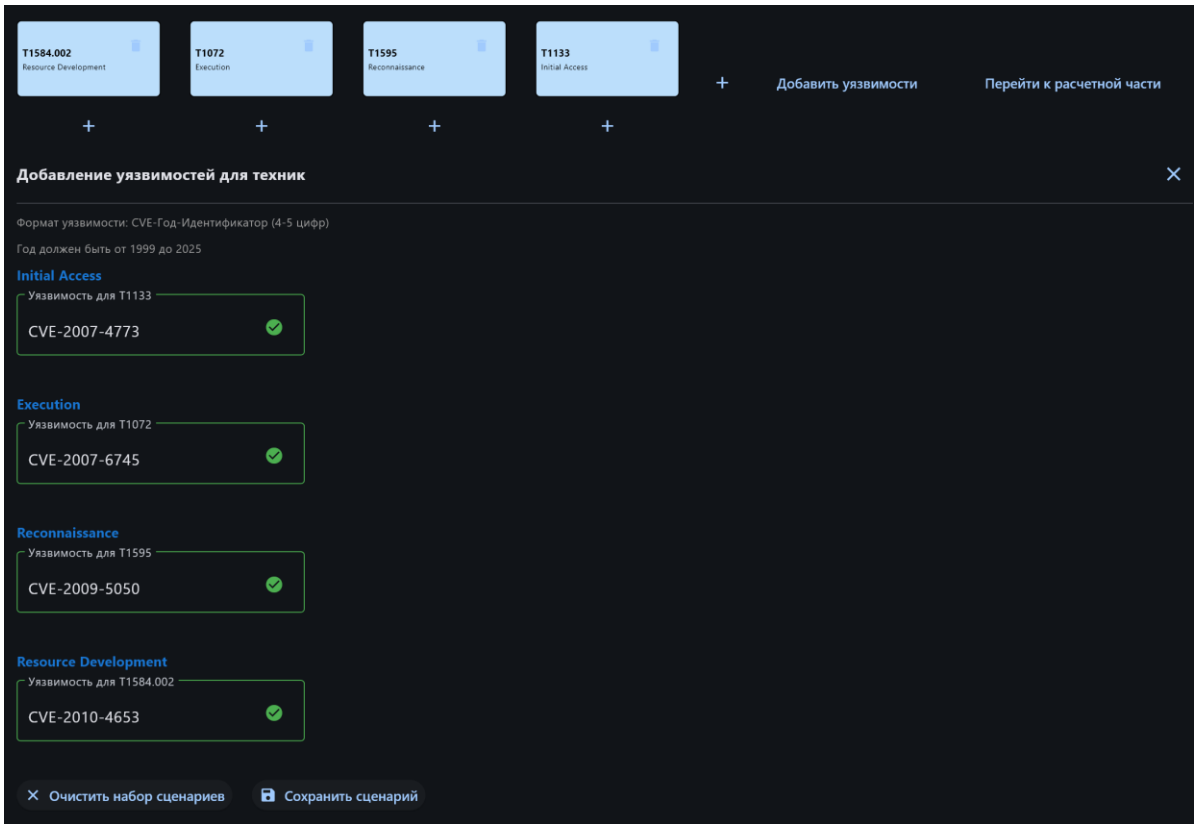


Рисунок 3.6 – Сборка вектора атаки

Во время сборки вектора атаки пользователь задает сначала тактики, а потом техники, через которые он хочет смоделировать сценарий. После выбора техник, для них пользователь задаст уязвимость, которую каждая из них будет эксплуатировать, когда выбор сделан, нужно сохранить файл. Затем в ранее показанном модуле с рисунка 4.5 пользователь перейдет во вкладку “Поток атак”, выберет созданный сценарий, и программа произведет расчеты.

Графическое отображение максимумов рисков для каждой уязвимости перенормированных вероятностей подчиняющиеся мультиномиальному распределению отображено на рисунке 3.7.

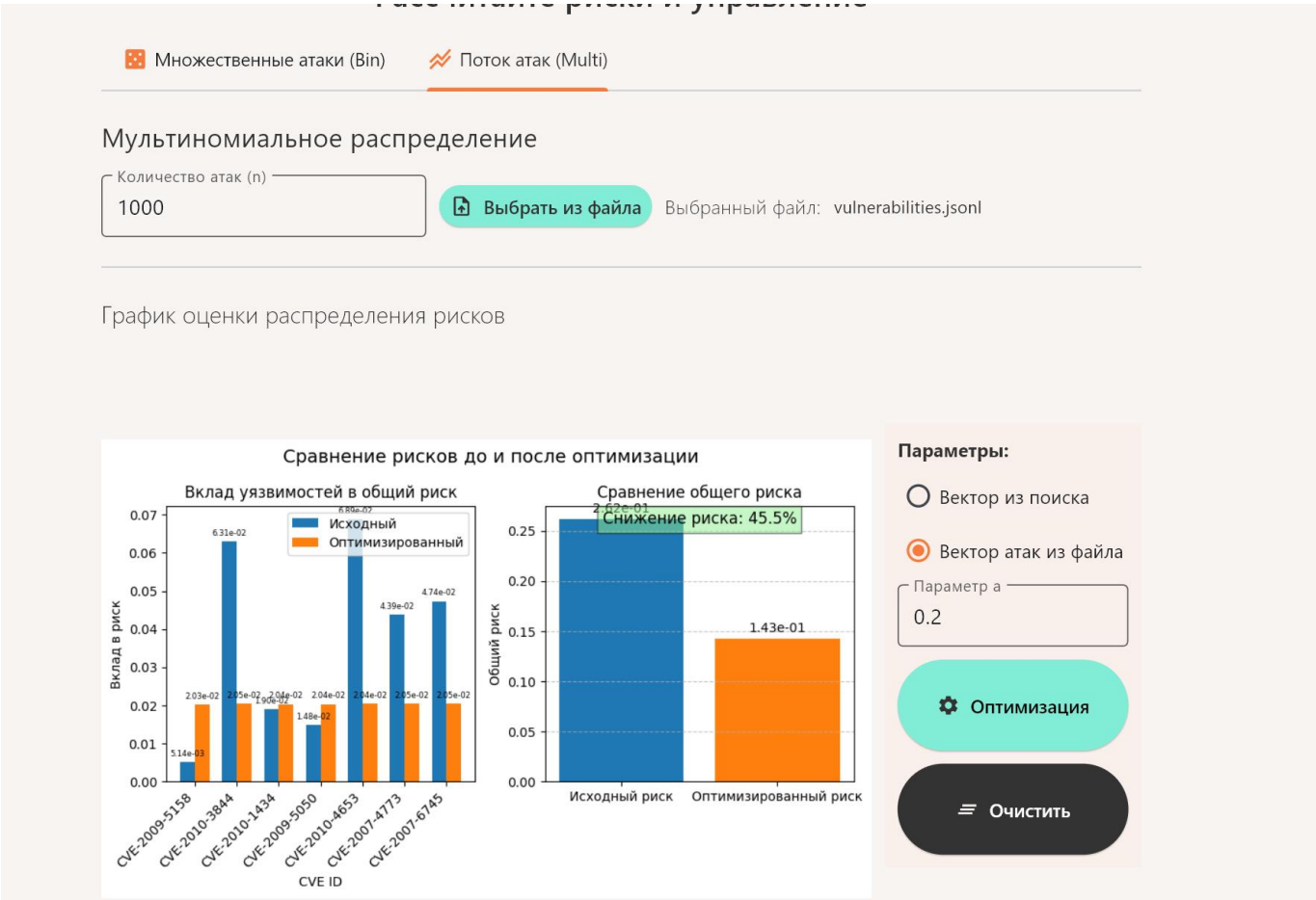


Рисунок 3.7 - Графическое отображение максимумов рисков для каждой уязвимости перенормированных вероятностей

Где a – это альфа, коэффициент потери ценности.

4. Программный код базовых модулей функции

1) Функция расчета риска для одиночных атак:

```
def calculate_risk(technique_id, techniques, cve_params):
    """Расчет риска для техники с новой структурой данных"""
    # Поиск техники по ID
    tech_entry = next((t for t in techniques if t['Technique_ID'] == technique_id),
None)
    if not tech_entry:
        return []
    # Сбор всех CVE для техники
    cves = []
    for cwe in tech_entry['CWEs']:
        cves.extend(cwe['CVEs'])
    cves = list(set(cves))
    if not cves:
        return []
    results = []
    for cve in cves:
        if cve not in cve_params:
            continue
        params = cve_params[cve]
        # Получение параметров с безопасным доступом (значение по умолчанию
0.0)
        confidentiality = params.get('confidentialityImpact', 0.0)
        integrity = params.get('integrityImpact', 0.0)
        availability = params.get('availabilityImpact', 0.0)
        cvss = params.get('CVSS3_base_score', 0.0)
        epss = params.get('EPSS', 0.0)
        weights = params.get('shapley_weights', {'C': 0.0, 'I': 0.0, 'A': 0.0})
```

```
        is_in_cisa_kev = params.get('is_in_cisa_kev', 0)
        # Извлечение весов
        c_weight = weights.get('C', 0.0)
        i_weight = weights.get('I', 0.0)
        a_weight = weights.get('A', 0.0)
        # Расчет суммы весов
        total_weight = c_weight + i_weight + a_weight
        if total_weight == 0:
            c_share = 0.0
            i_share = 0.0
            a_share = 0.0
        else:
            c_share = c_weight / total_weight
            i_share = i_weight / total_weight
            a_share = a_weight / total_weight
        # Расчет рисков по новым формулам
        risk_conf = epss * cvss * confidentiality * c_share * is_in_cisa_kev * (1/35)
        risk_integ = epss * cvss * integrity * i_share * is_in_cisa_kev * (1/35)
        risk_avail = epss * cvss * availability * a_share * is_in_cisa_kev * (1/35)
        # Суммарный риск
        total_risk = risk_conf + risk_integ + risk_avail
        results.append({
            'CVE': cve,
            'Риск': total_risk,
            'Риск конфиденциальности': risk_conf,
            'Риск целостности': risk_integ,
            'Риск доступности': risk_avail
        })
    return sorted(results, key=lambda x: x['Риск'], reverse=True)
```

2) Функция перенормировки вероятности для достижения нахождения квантиля распределения в заданной позиции:

```
def scale_probabilities(cve_list: List[Dict], n: int,
                        target_quantile: float = 0.38,
                        center:          float = 0.55,
                        width:           float = 0.4) -> List[Dict]:
    """
    Перенормирует вероятности так, чтобы квантиль распределения находился в заданной позиции.
        Параметры:
        cve_list: список CVE
        n: количество атак
        target_quantile: целевой квантиль (0.8 = 80%-й)
        center: центр для пиков (0.0-1.0)
        width: ширина разброса пиков (0.0-1.0)
            Возвращает:
        Новый список CVE с ключом 'p_scaled'
    """
    new_cves = []
    for cve in cve_list:
        p_orig = cve['p']
        # Рассчитываем квантиль для исходного распределения
        k_quantile = binom.ppf(target_quantile, n, p_orig)
        # Целевая позиция квантиля (в диапазоне [center-width/2, center+width/2])
        target_position = center - width/2 + width * (p_orig - np.min([p['p'] for p in cve_list])) / (
            np.max([p['p'] for p in cve_list]) - np.min([p['p'] for p in cve_list]) + 1e-9)
            # Ограничиваем целевую позицию
        target_position = max(0.05, min(0.95, target_position))
        # Вычисляем новую вероятность
```

```
            p_scaled = target_position * (k_quantile / (n * target_quantile))
            # Гарантируем корректные границы вероятности
            p_scaled = max(0.001, min(0.999, p_scaled))
            new_cve = cve.copy()
            new_cve['p'] = p_scaled
            new_cves.append(new_cve)
        return new_cves
```

3) Функция вычисления вектора рисков для всех k от 0 до n

```
    def calculate_binomial_risk_vector(u_i: float, n: int, p: float) -> Tuple[np.ndarray, np.ndarray]:
        """
        Вычисляет вектор рисков для всех k от 0 до n
        Параметры:
            u_i (float): ущерб при успешной атаке
            n (int): общее количество атак
            p (float): вероятность успешной атаки
        Возвращает:
            Tuple[np.ndarray, np.ndarray]: (k_values, risk_values)
        """
        k_values = np.arange(0, n + 1)
        risk_values = np.zeros_like(k_values, dtype=float)
            for i, k in enumerate(k_values):
            if k == 0:
                risk_values[i] = 0.0
            else:
                log_comb = math.lgamma(n + 1) - math.lgamma(k + 1) - math.lgamma(n - k + 1)
                log_p = k * math.log(p) + (n - k) * math.log(1 - p)
                risk_values[i] = k * u_i * math.exp(log_comb + log_p)
```

```
    return k_values, risk_values
```

4) Функция вычисления риска для всех CVE как по отдельности, так и вместе взятых:

```
def calculate_all_risks(cve_list: List[dict], n: int) -> Tuple[List[Tuple[np.ndarray, np.ndarray, str]], np.ndarray, np.ndarray]:
    """
    Вычисляет риски для всех CVE и суммарный риск
    Параметры:
        cve_list: список словарей CVE с ключами 'u_i', 'p', 'label'
        n: общее количество атак
    Возвращает:
        all_risks: список (k_values, risk_values, label) для каждой CVE
        total_risk: суммарный риск по всем CVE
        k_values: общие значения k
    """
    all_risks = []
    total_risk = np.zeros(n + 1)

    for cve in cve_list:
        k_values, risk_values = calculate_binomial_risk_vector(cve['u_i'], n, cve['p'])
        label = cve.get('label', '')
        all_risks.append((k_values, risk_values, label))
        total_risk += risk_values

    return all_risks, total_risk, k_values
```

5) Функция выравнивания пиков рисков для всех CVE до общего максимума:

```
def adjust_risks(all_risks: List[Tuple[np.ndarray, np.ndarray, str]]) -> Tuple[List[Tuple[np.ndarray, np.ndarray, str, np.ndarray]], np.ndarray]:
```

```
    """
    Выравнивает пики рисков для всех CVE до общего максимума
    Параметры:
        all_risks: список кортежей (k_values, risk_values, label)
    Возвращает:
        adjusted_risks: список (k_values, original_risk, label, adjusted_risk)
        adjusted_total_risk: новый суммарный риск
    """
    all_max_risks = [np.max(risk_vals) for _, risk_vals, _ in all_risks]
    R_max = max(all_max_risks)

    adjusted_risks = []
    adjusted_total_risk = None
    for k_vals, risk_vals, label in all_risks:
        current_max = np.max(risk_vals)
        e = R_max / current_max if current_max > 0 else 1.0
        adjusted_risk = risk_vals * e
        adjusted_risks.append((k_vals, risk_vals, label, adjusted_risk))
        if adjusted_total_risk is None:
            adjusted_total_risk = np.zeros_like(adjusted_risk)
        adjusted_total_risk += (adjusted_risk * 1.2)
    return adjusted_risks, adjusted_total_risk
```

6) Функция расчета мультиномиального распределения:

```
def calculate_multinomial_prob(n: int, x: List[int], p: List[float]) -> float:
    """
    Вычисляет вероятность мультиномиального распределения
        Параметры:
        n: общее количество атак
        x: список успешных атак по каждой уязвимости [x1, x2, ..., xm]
```

```
    p: список вероятностей успеха [p1, p2, ..., pm] (сумма=1)
Возвращает:
    float: вероятность P(x1,...,xm)
"""
# Проверка сумм
if abs(sum(p) - 1.0) > 1e-6:
    raise ValueError("Сумма вероятностей p_i должна быть равна 1")
if sum(x) != n:
    raise ValueError(f"Сумма x_i ({sum(x)}) должна равняться n ({n})")
# Логарифм факториала n
log_n_fact = math.lgamma(n + 1)
    # Сумма логарифмов факториалов x_i
log_x_fact_sum = sum(math.lgamma(x_i + 1) for x_i in x)
    # Сумма x_i * ln(p_i)
log_p_sum = sum(x_i * math.log(p_i) for x_i, p_i in zip(x, p) if p_i > 0)
    # Итоговая вероятность
return math.exp(log_n_fact - log_x_fact_sum + log_p_sum)
```

7) Функция вычисления риска многовекторной атаки:

```
def calculate_multinomial_risk(
    cve_data: List[Dict],
    n: int,
    alpha: float,  # Единый коэффициент для всех уязвимостей
    total_value: Optional[float] = None ) -> Tuple[float, List[float]]:
    """
    Вычисляет риск для многовекторной атаки
        Параметры:
        cve_data: список словарей с ключами:
            'p': вероятность успеха (EPSS)
            'C_i': ценность ресурса
```

```
    n: общее количество атак
    alpha: единый коэффициент потери ценности для всех CVE
    total_value: общая ценность (для нормировки, опционально)
Возвращает:
    Tuple[float, List[float]]: суммарный риск и вклады по CVE
"""
m = len(cve_data)
if m == 0:
    return 0.0, []
# Извлекаем параметры
p_vals = [cve['p'] for cve in cve_data]
C_vals = [cve['C_i'] for cve in cve_data]
# Нормировка общей ценности
if total_value is not None:
    current_total = sum(C_vals)
    if current_total > 0:
        C_vals = [C * total_value / current_total for C in C_vals]
# Нормируем вероятности
total_p = sum(p_vals)
p_normalized = [p_i / total_p for p_i in p_vals]
# Рассчитываем ожидаемое количество успешных атак
x_vals = [max(0, min(n, round(n * p_i))) for p_i in p_normalized]
# Корректируем сумму x_i
diff = n - sum(x_vals)
if diff != 0:
    max_idx = np.argmax(p_normalized)
    x_vals[max_idx] += diff
# Рассчитываем ущерб u_i = alpha * C_i
u_vals = [alpha * C_i for C_i in C_vals]  # Простое умножение на alpha
# Суммарный ущерб
```

```
    total_damage = sum(x_i * u_i for x_i, u_i in zip(x_vals, u_vals))
    # Вероятность
    prob = calculate_multinomial_prob(n, x_vals, p_normalized)
    risk_contributions = [x_i * u_i * prob for x_i, u_i in zip(x_vals, u_vals)]
    return total_damage * prob, risk_contributions
```

8) Функция оптимизации распределение ресурсов для минимизации риска:

```
def optimize_resources(
    cve_data: List[Dict],
    n: int,
    alpha: float,  # Единый коэффициент для всех
    total_value: float) -> Tuple[List[float], float, List[float]]:
    """
    Оптимизирует распределение ресурсов для минимизации риска
        Параметры:
        cve_data: список словарей с ключами:
            'p': вероятность успеха (EPSS)
            'C_i': исходная ценность ресурса
        n: общее количество атак
        alpha: единый коэффициент потери ценности
        total_value: суммарная ценность всех ресурсов
        Возвращает:
        Tuple[List[float], float, List[float]]: оптимальные C_i, минимальный риск, вклады по CVE
    """
    m = len(cve_data)
    if m == 0:
        return [], 0.0, []
    # Рассчитываем веса (w_i = 1 / (n * p_i * alpha))
    # alpha одинаков для всех, поэтому влияет только на масштаб
```

```
weights = []
for cve in cve_data:
    p_i = cve['p']
    if p_i > 0:
        weights.append(1 / (n * p_i * alpha))
    else:
        weights.append(0)
    # Если все веса нулевые, распределяем равномерно
if sum(weights) <= 0:
    return [total_value / m] * m, 0.0, []
    # Оптимальные C_i пропорциональны весам
C_opt = [total_value * w / sum(weights) for w in weights]
    # Рассчитываем минимальный достижимый риск
optimized_data = []
for i, cve in enumerate(cve_data):
    optimized_data.append({
        'p': cve['p'],
        'C_i': C_opt[i]
    })
    # Рассчитываем минимальный достижимый риск и вклады
min_risk, risk_contributions = calculate_multinomial_risk(
    optimized_data,
    n,
    alpha,
    total_value
)
```